

RECURSION

ISIS 4217

Sum squares of a list

Sum squares of a list

```
local SumSqr in
   fun {SumSqr L}
      local Sum X in
         Sum = {NewCell 0}
         for  X in 1..{Length L} do
          Sum := @Sum + {Nth L X}*{Nth L X}
         end
         @Sum
      end
   end
   {Show {SumSqr [1 2 3 4 5]}}
end
```

1. Sum squares of a list

Do it yourself

1.

Do it yourself

```
[1 2 3 4 5]
```

1.

Do it yourself

```
[1 2 3 4 5]
```

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then Car*Car + {SumSqr Cdr}
   end
end
```

1.

Do it yourself

```
[1 2 3 4 5]
```

```
fun {SumSqr Lst}
   local SumSqr2 in
      fun {SumSqr2 Lst Accum}
         case Lst of nil then Accum
         [] Car|Cdr then {SumSqr2 Cdr Accum+Car*Car}
         end
      end
   {SumSqrt2 Lst 0}
   end
end
```

1.

Do it yourself

```
[1 2 3 4 5]
```

```
fun {SumSqr Lst}
   local SumSqr2 in
      fun {SumSqr2 Lst Accum}
         case Lst of nil then Accum
         [] Car|Cdr then {SumSqr2 Cdr Accum+Car*Car}
         end
      end
   {SumSqrt2 Lst 0}
   end
end
```

better?

WHAT IS THE DIFFERENCE BETWEEN THE TWO SOLUTIONS

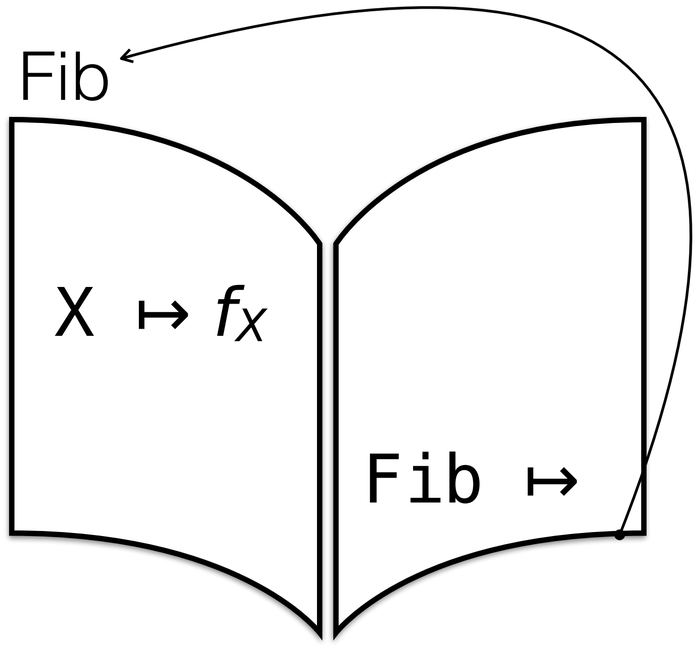


Recursion

A recursive **function** is defined in terms of itself

$$f_n = \begin{cases} 1 & \text{if } n <= 1 \\ f_{n-1} + f_{n-2} & \text{otherwise} \end{cases}$$

```
fun {Fib X}
   if X <= 1 then 1
   else {Fib X-1}+{Fib X-2}
   end
end
```

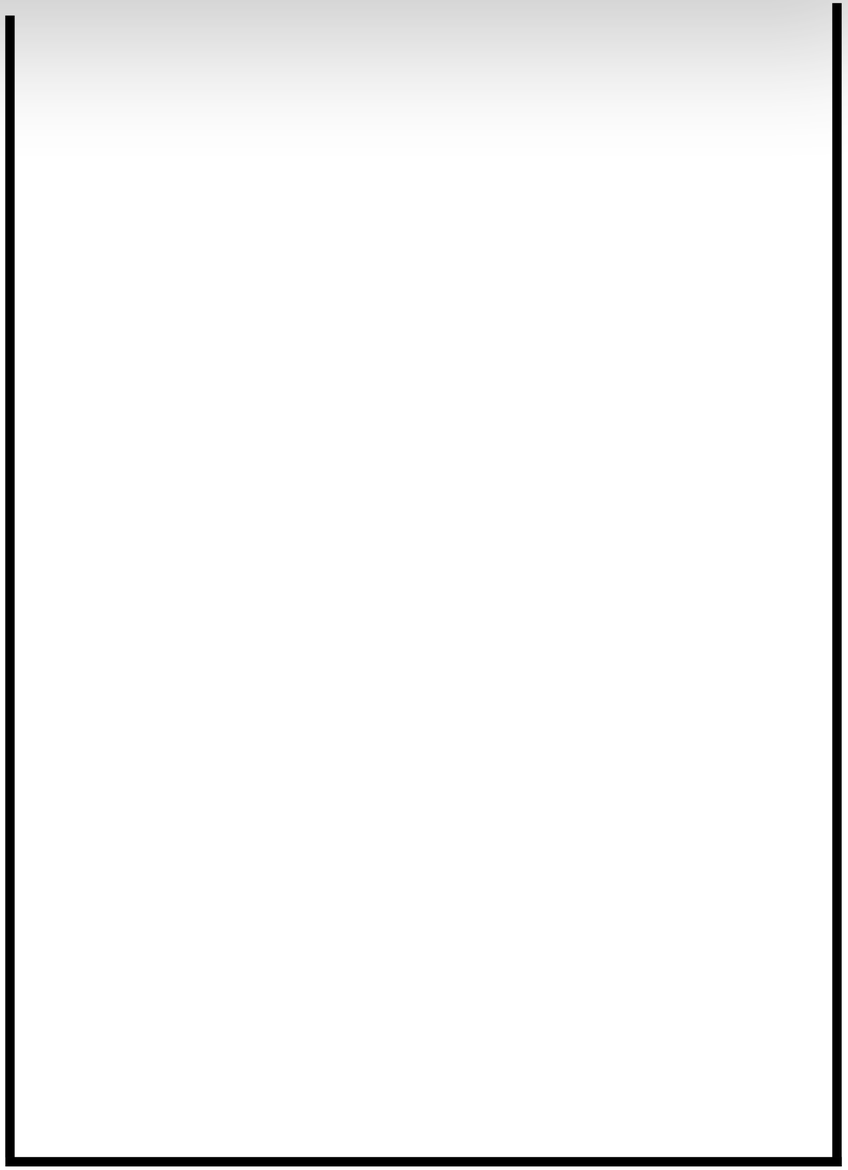


All about the stack

Solution 1

```
fun {SumSqr Lst}
  case Lst of nil then 0
  [] Car|Cdr then
       Car*Car+{SumSqr Cdr}
  end
end
{SumSqr [1 2 3]}
```

Stack

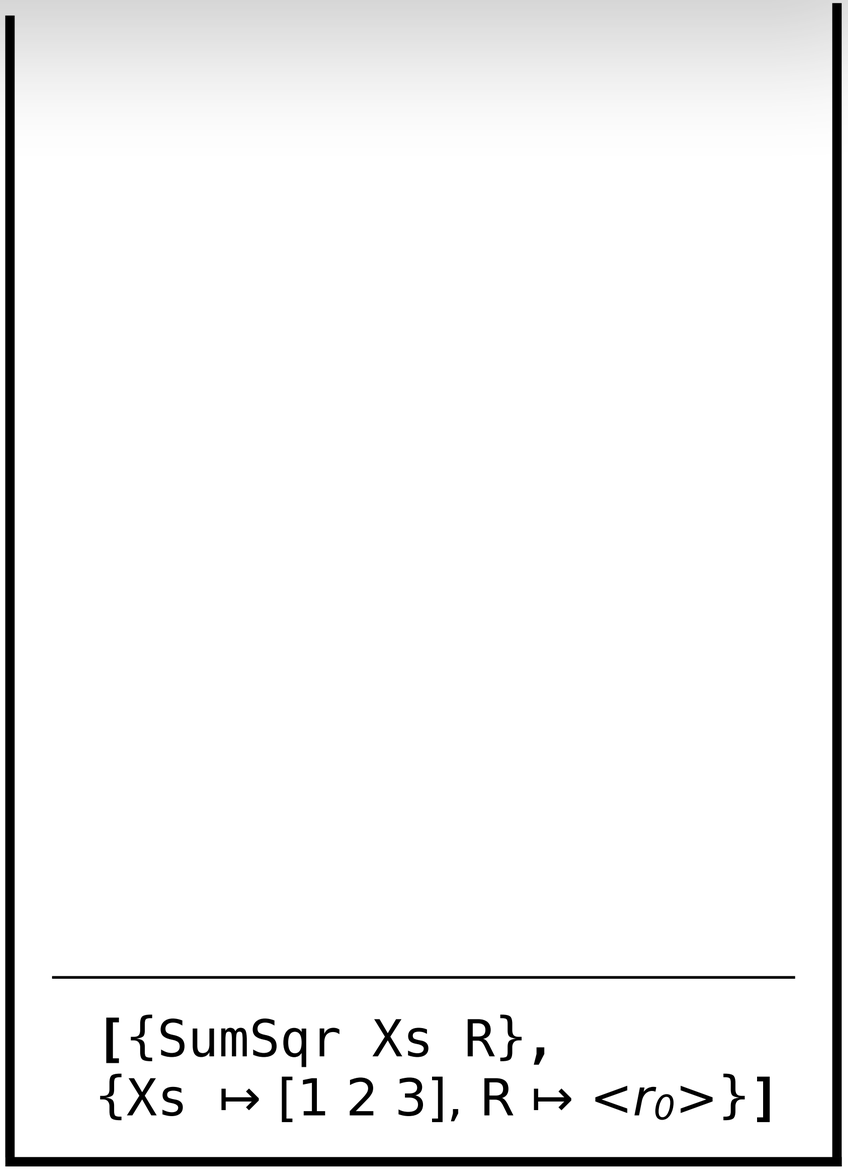


All about the stack

Solution 1

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```

Stack

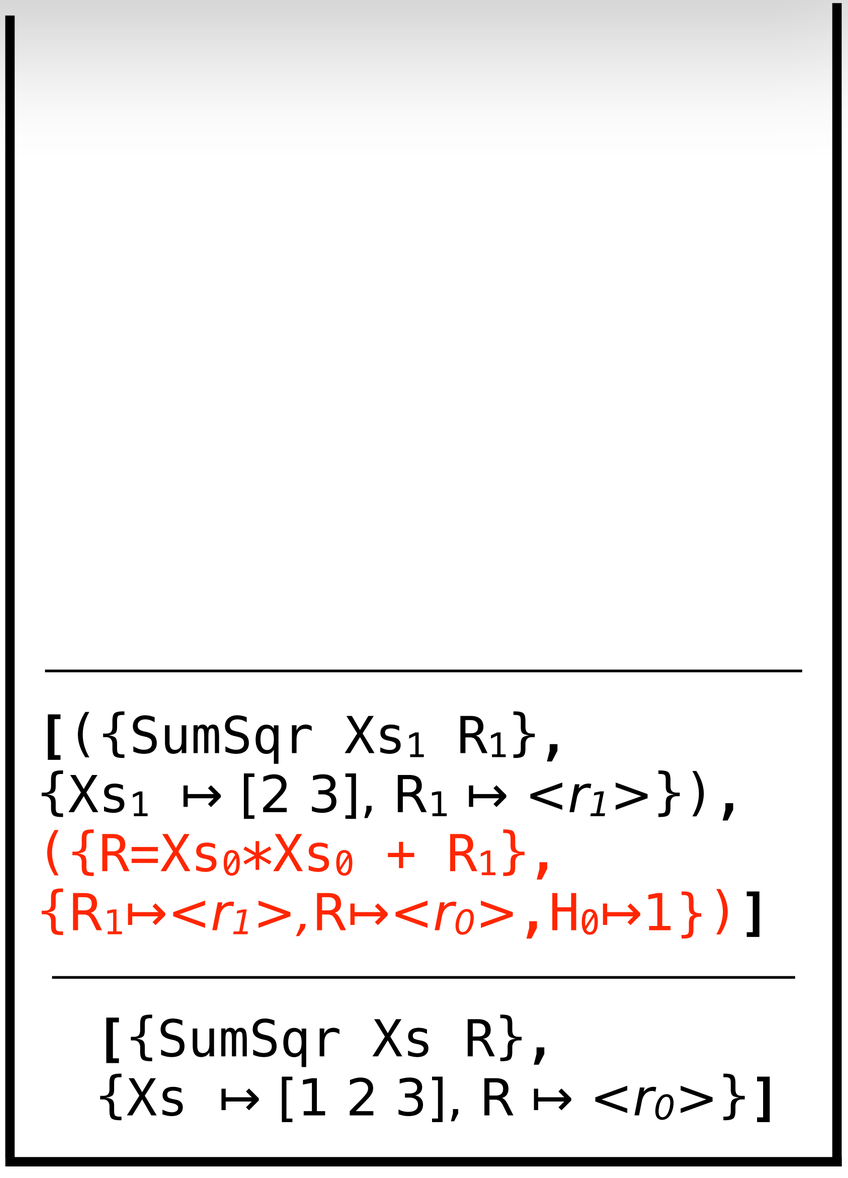


All about the stack

Solution 1

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```

Stack

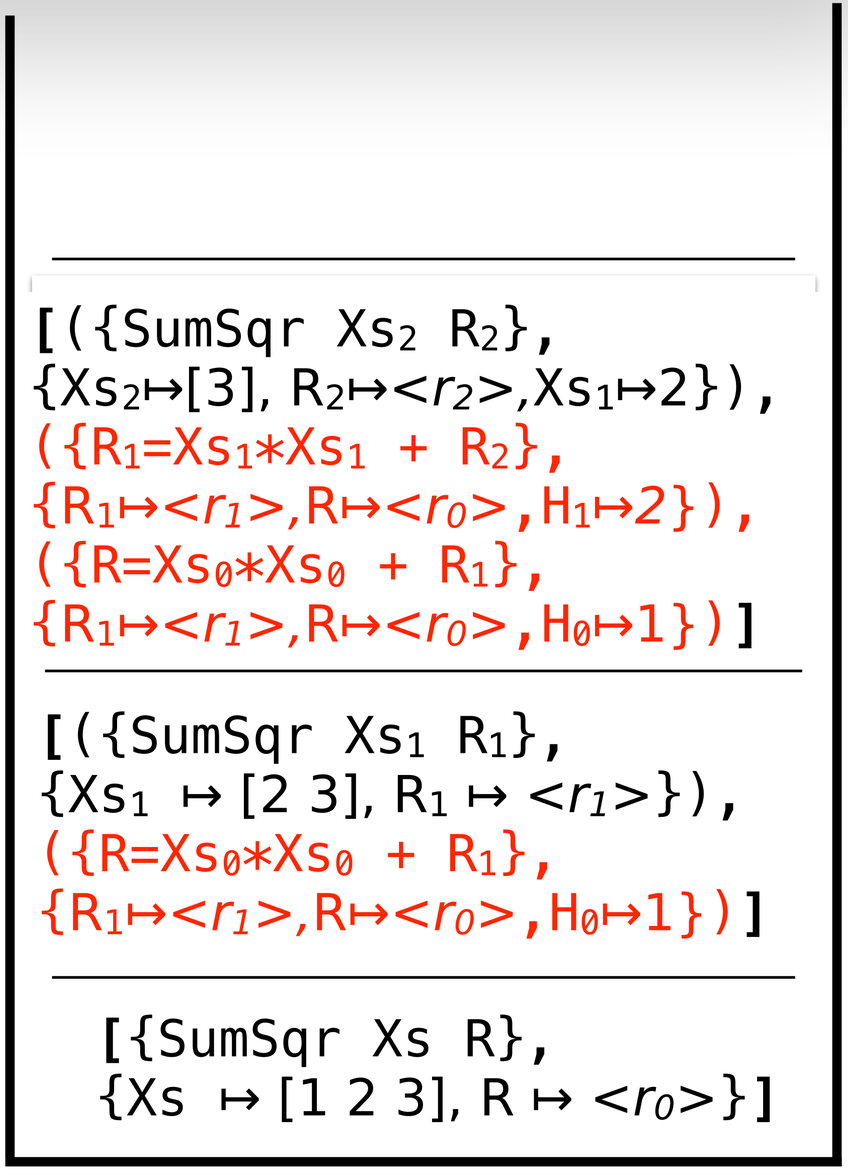


All about the stack

Solution 1

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```

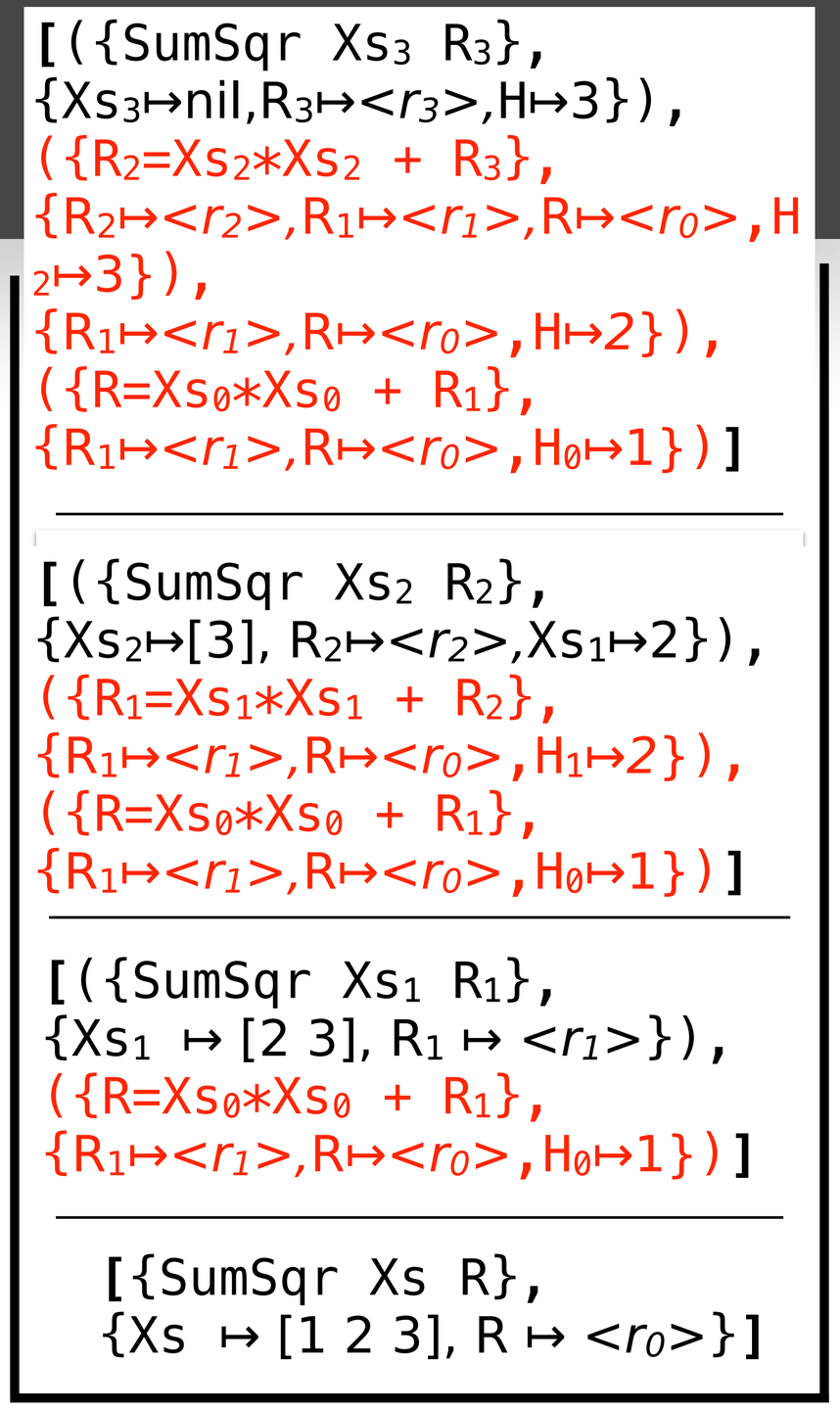
Stack



All about the stack

Solution 1

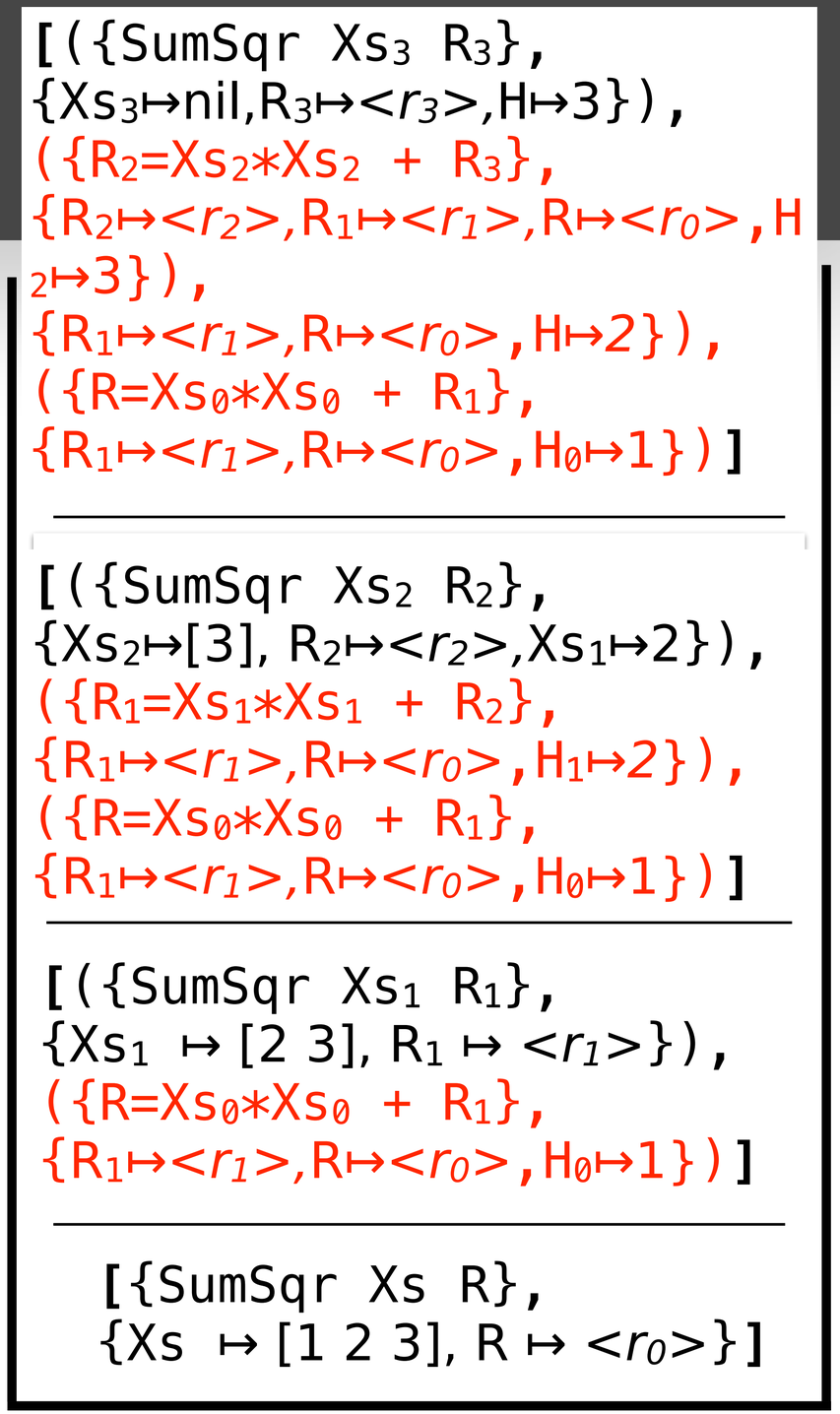
```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```



All about the stack

Solution 1

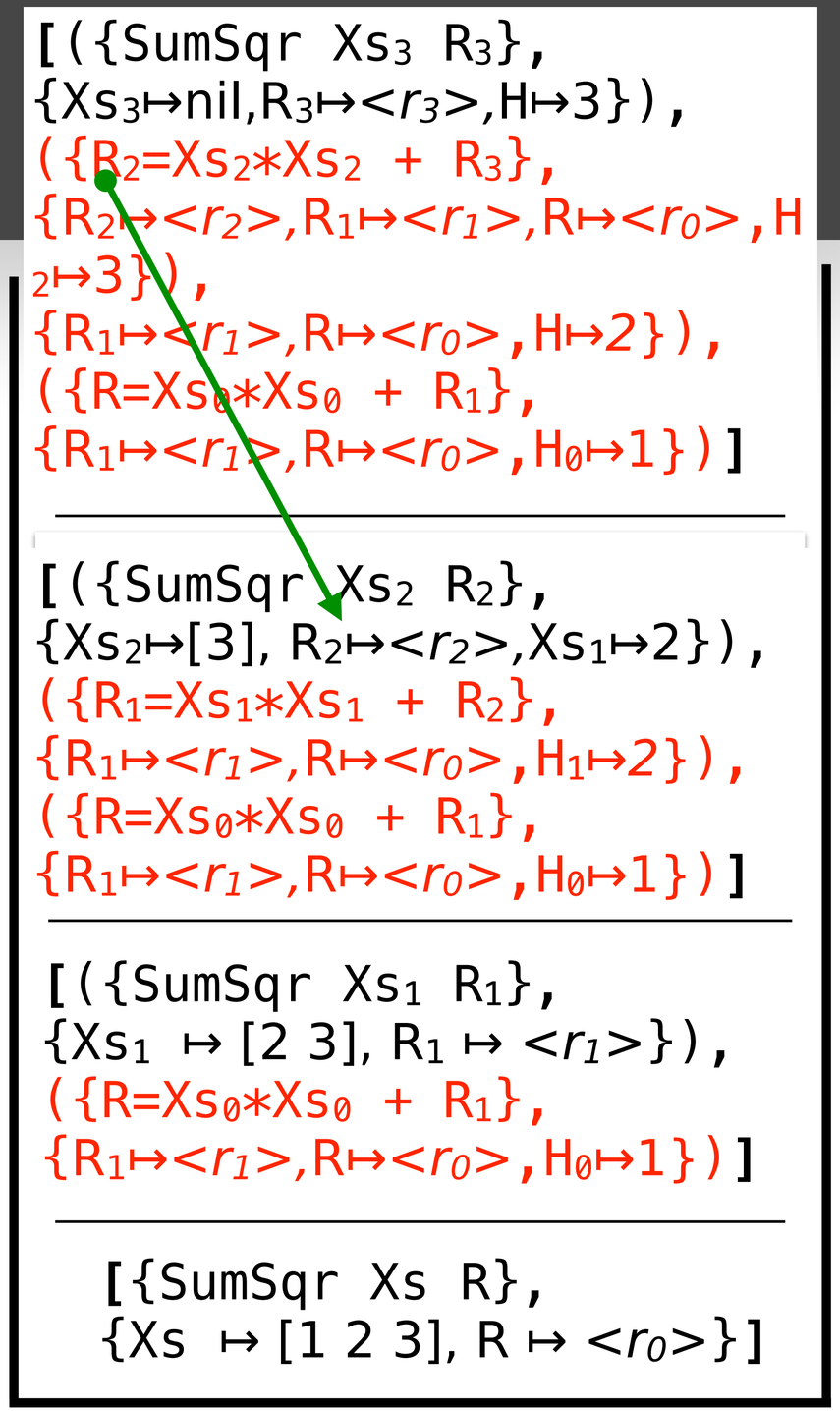
```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```



All about the stack

Solution 1

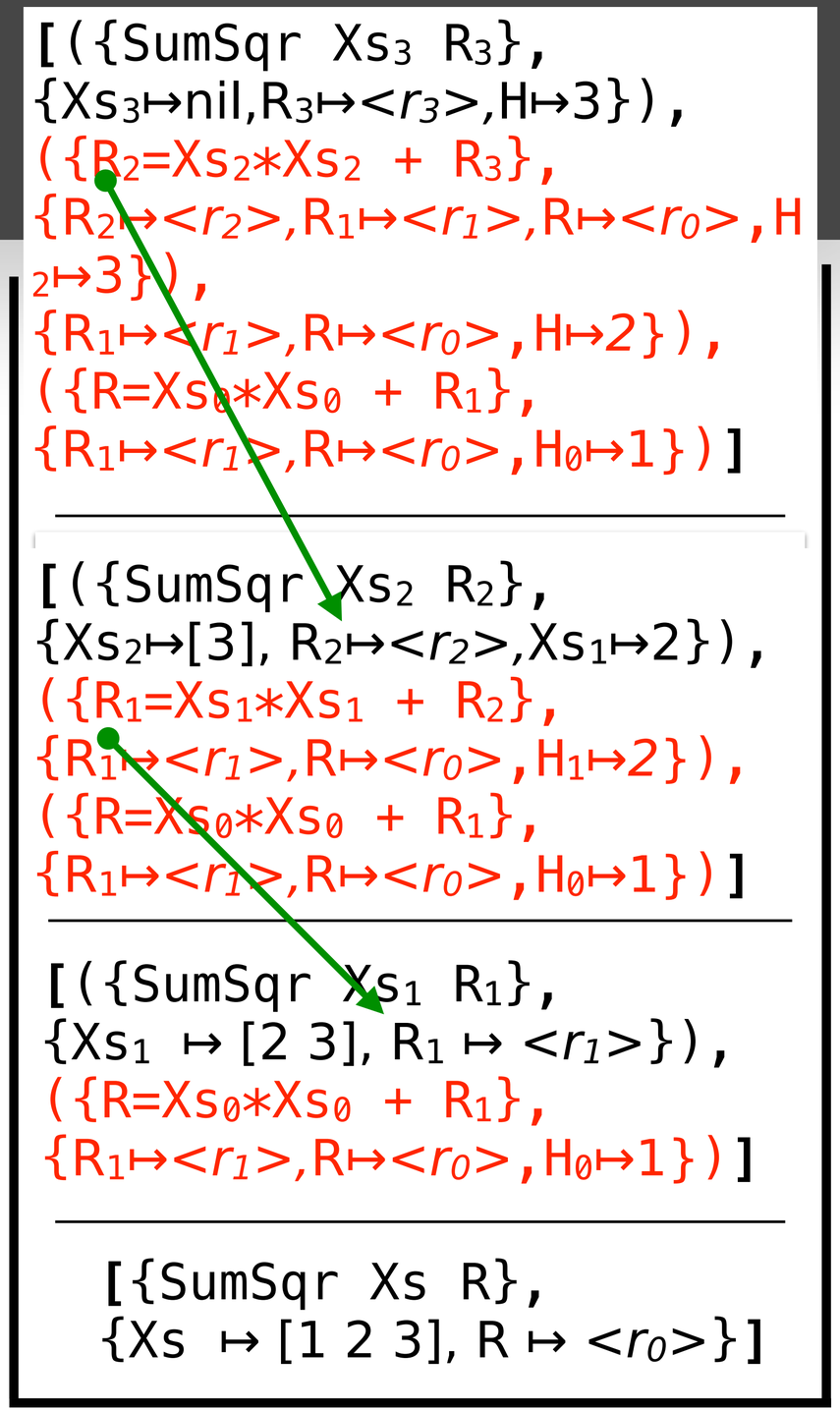
```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```



All about the stack

Solution 1

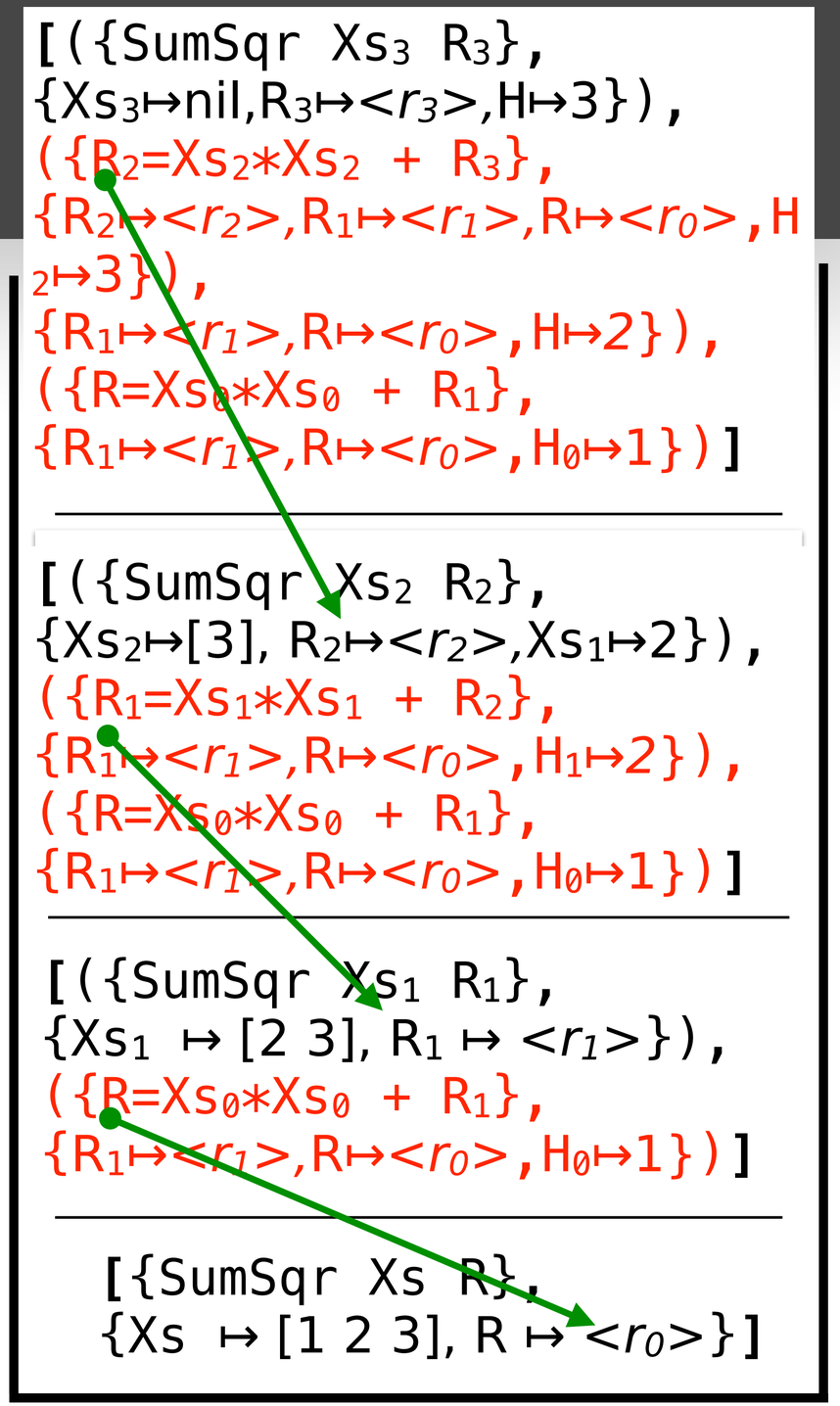
```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```



All about the stack

Solution 1

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```

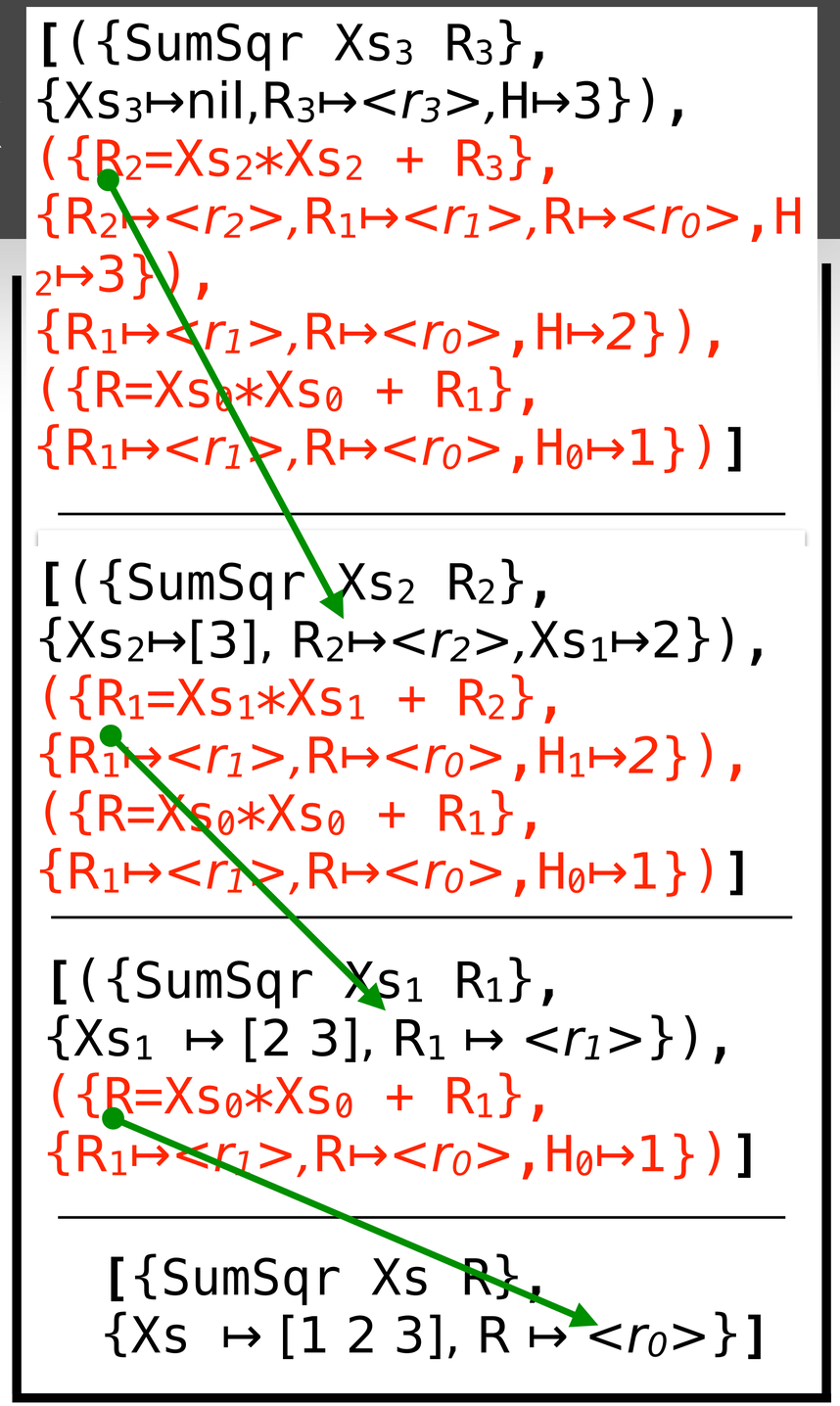


All about the stack

Solution 1

```
fun {SumSqr Lst}
   case Lst of nil then 0
   [] Car|Cdr then
         Car*Car+{SumSqr Cdr}
   end
end
{SumSqr [1 2 3]}
```

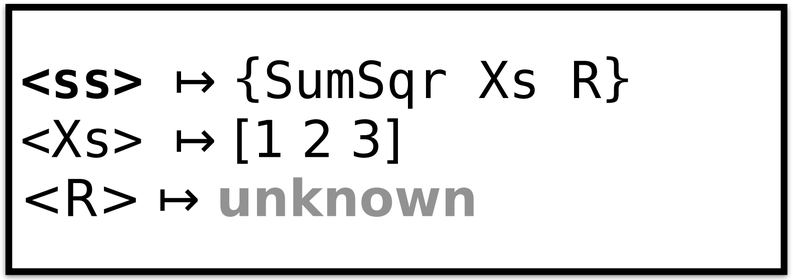
dangling reference across stack frames



Stack

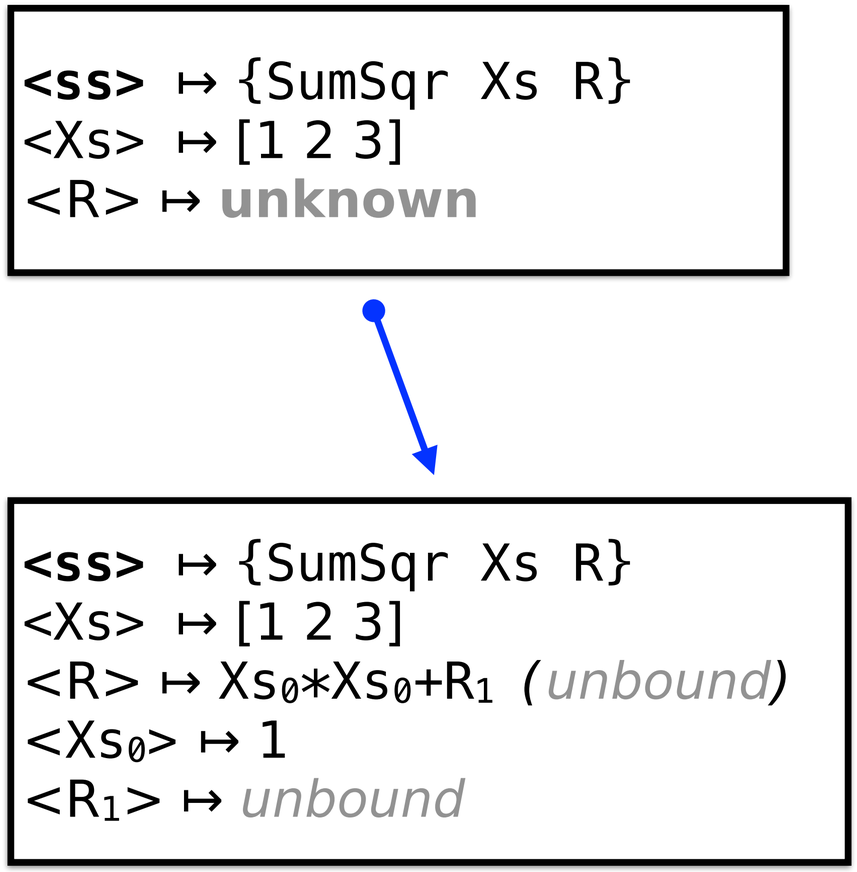
All about the stack

Solution 1



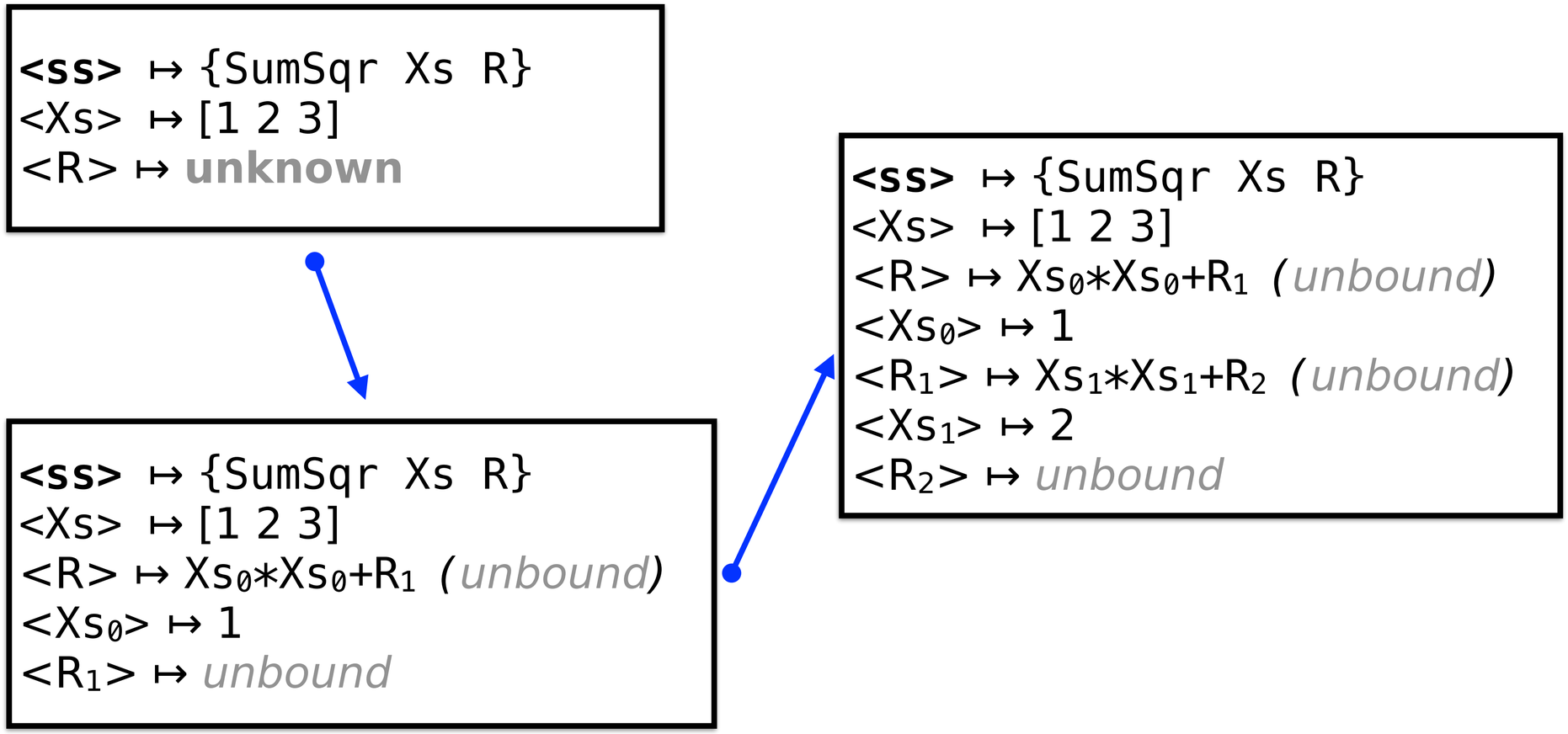
All about the stack

Solution 1



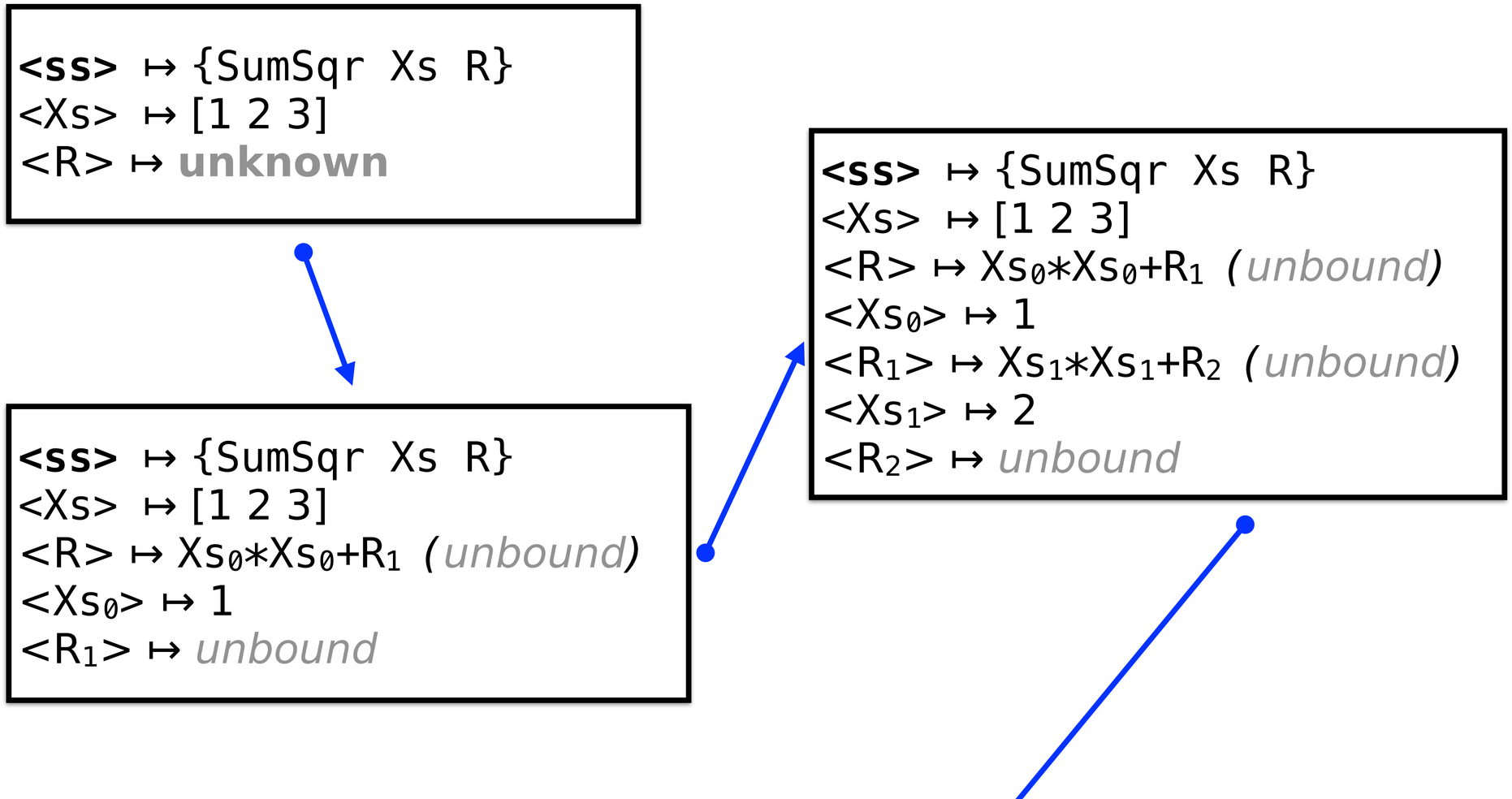
All about the stack

Solution 1



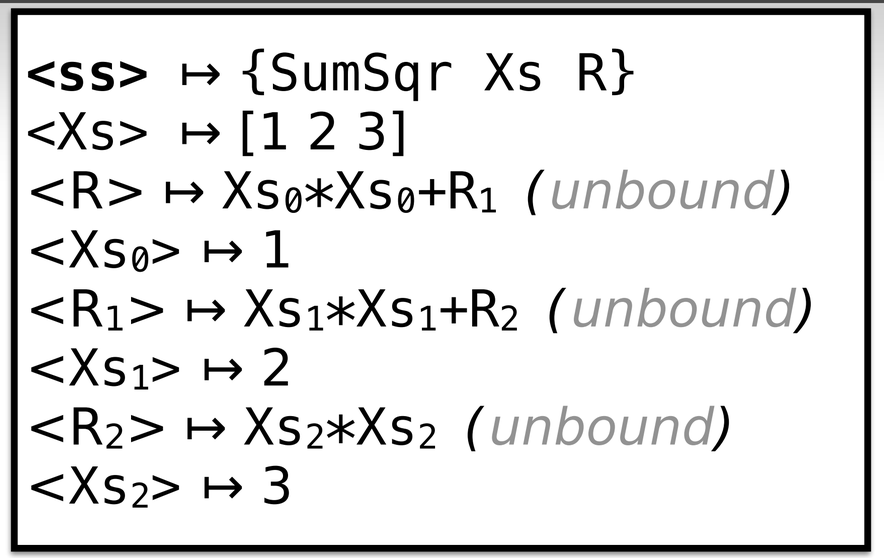
All about the stack

Solution 1



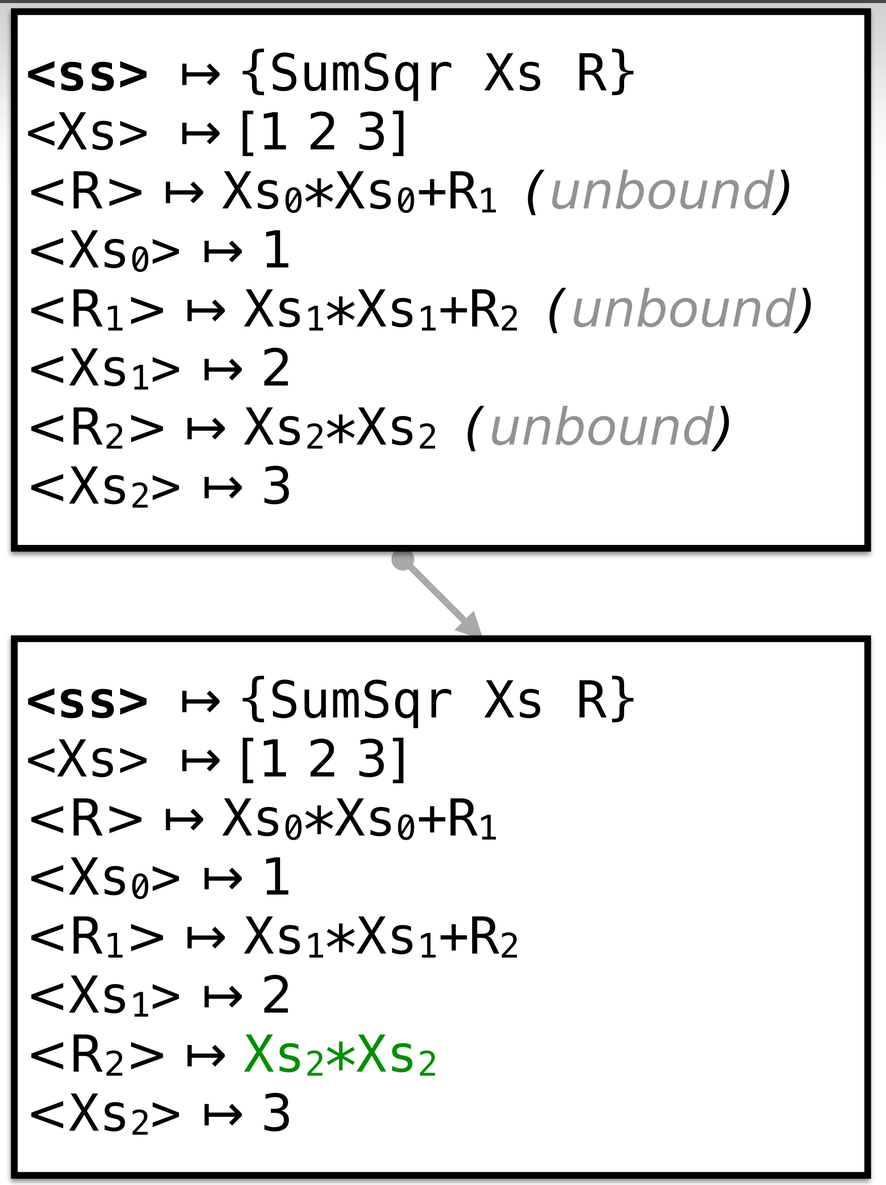
All about the stack

Solution 1



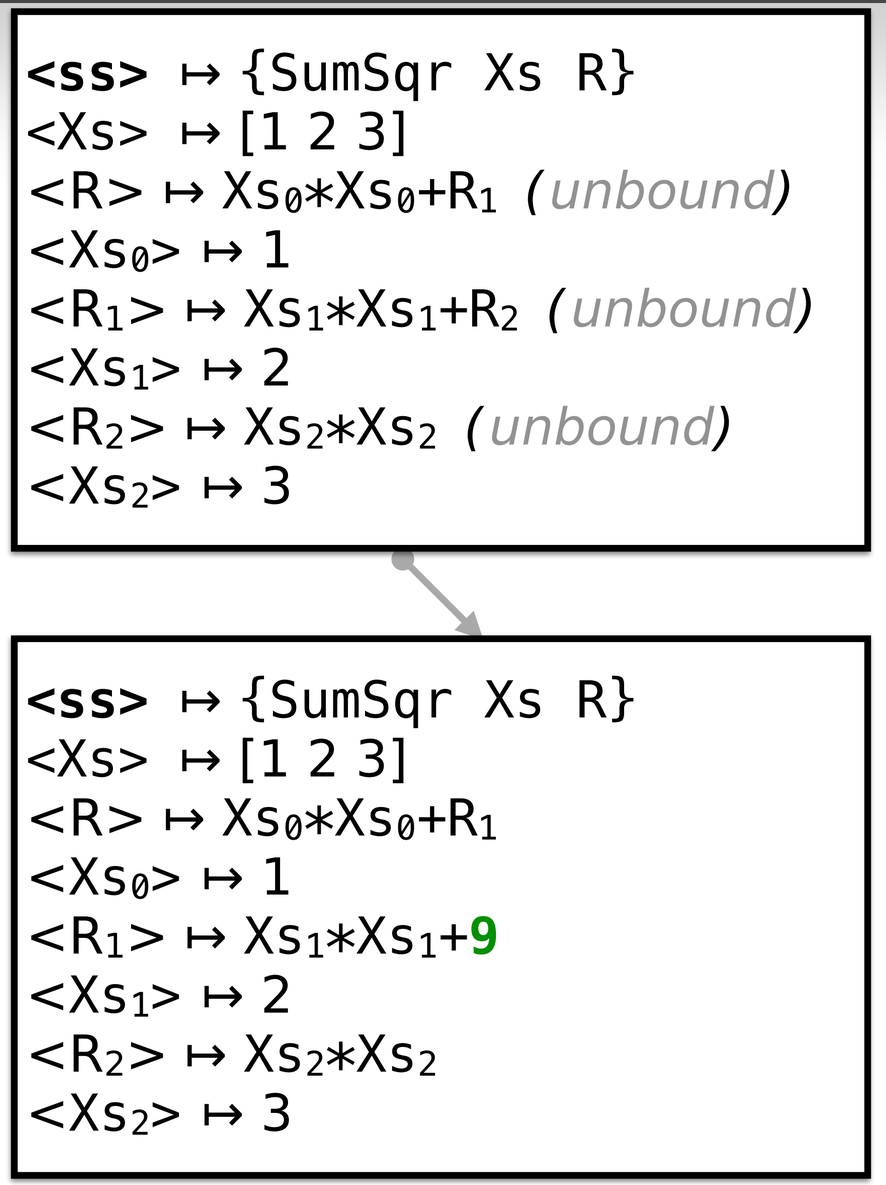
All about the stack

Solution 1



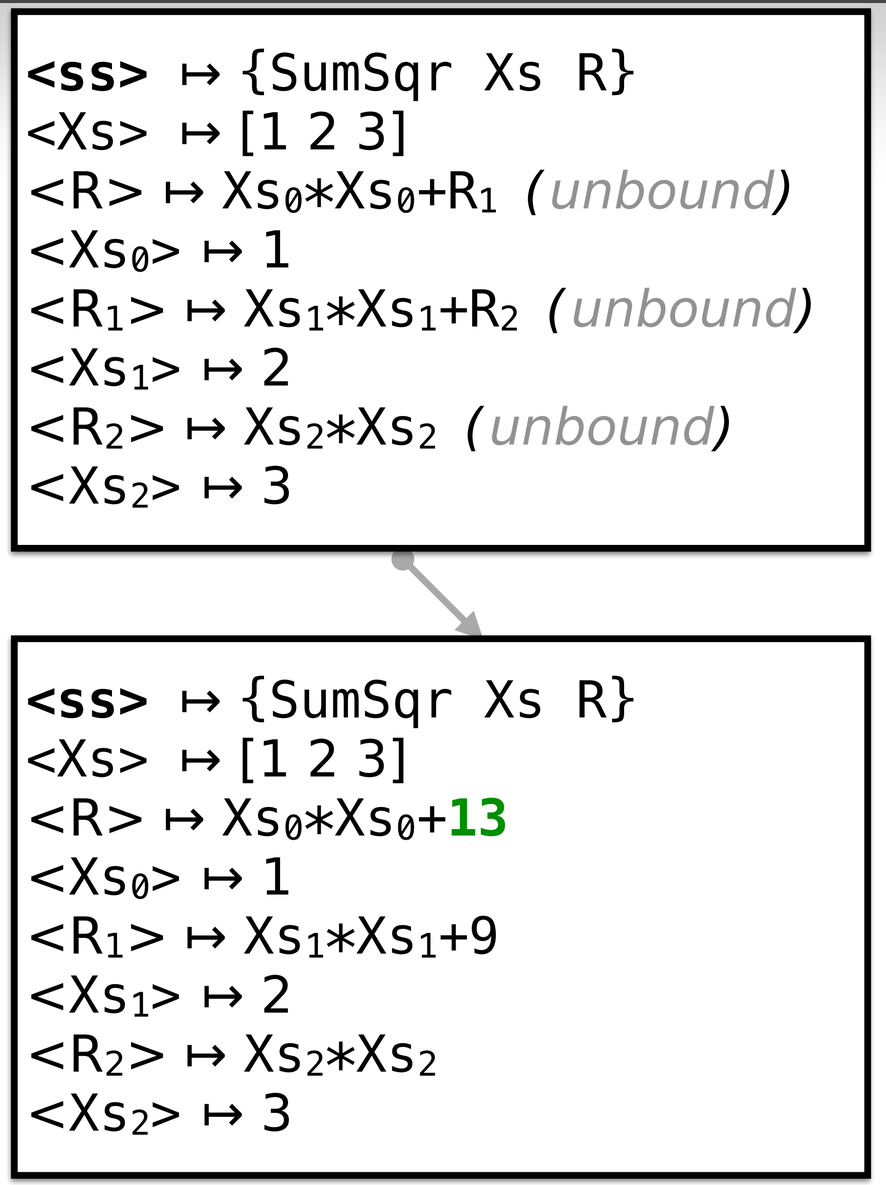
All about the stack

Solution 1



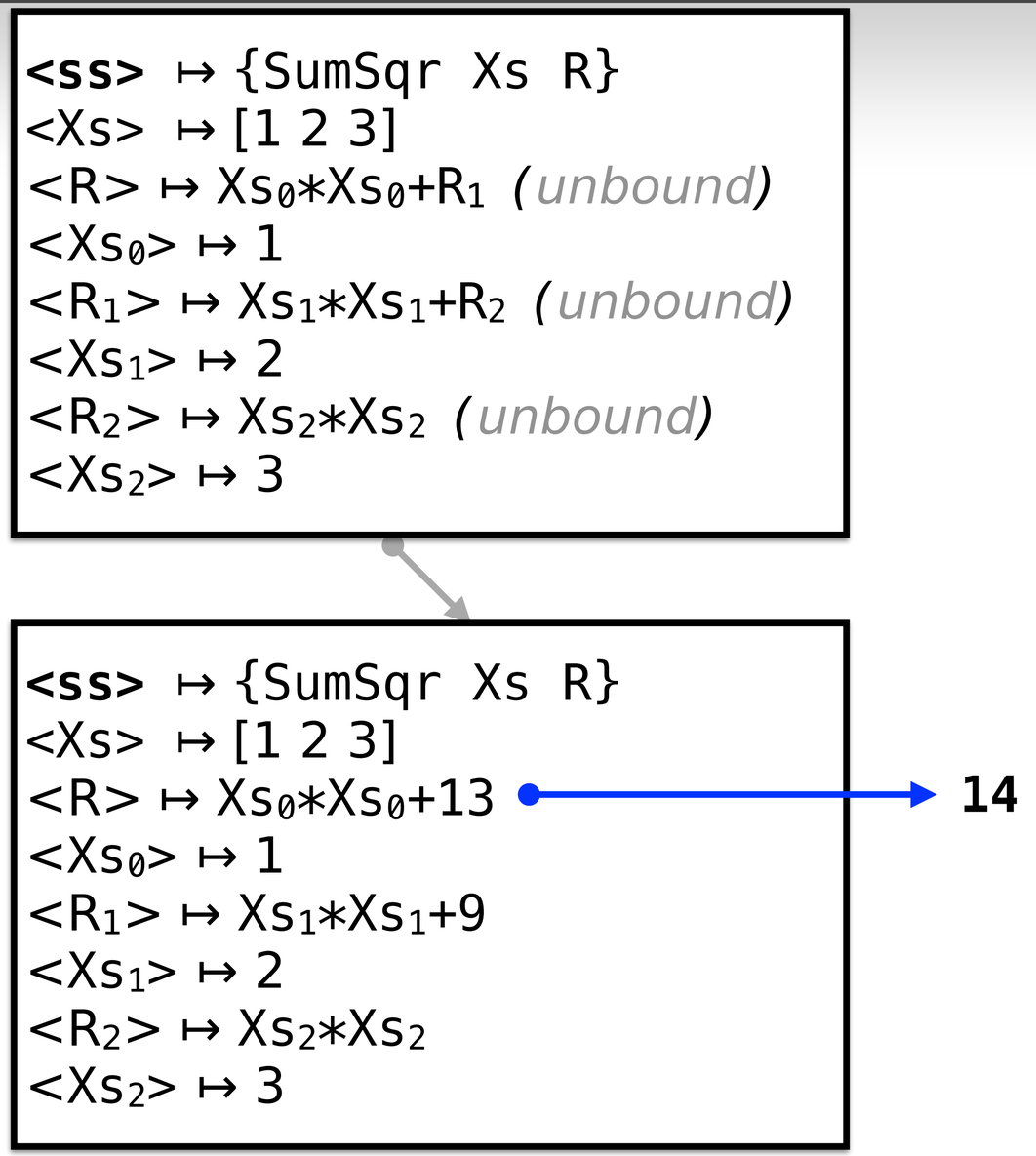
All about the stack

Solution 1



All about the stack

Solution 1



TOTAL NUMBER OF
FRAMES THAT I NEED

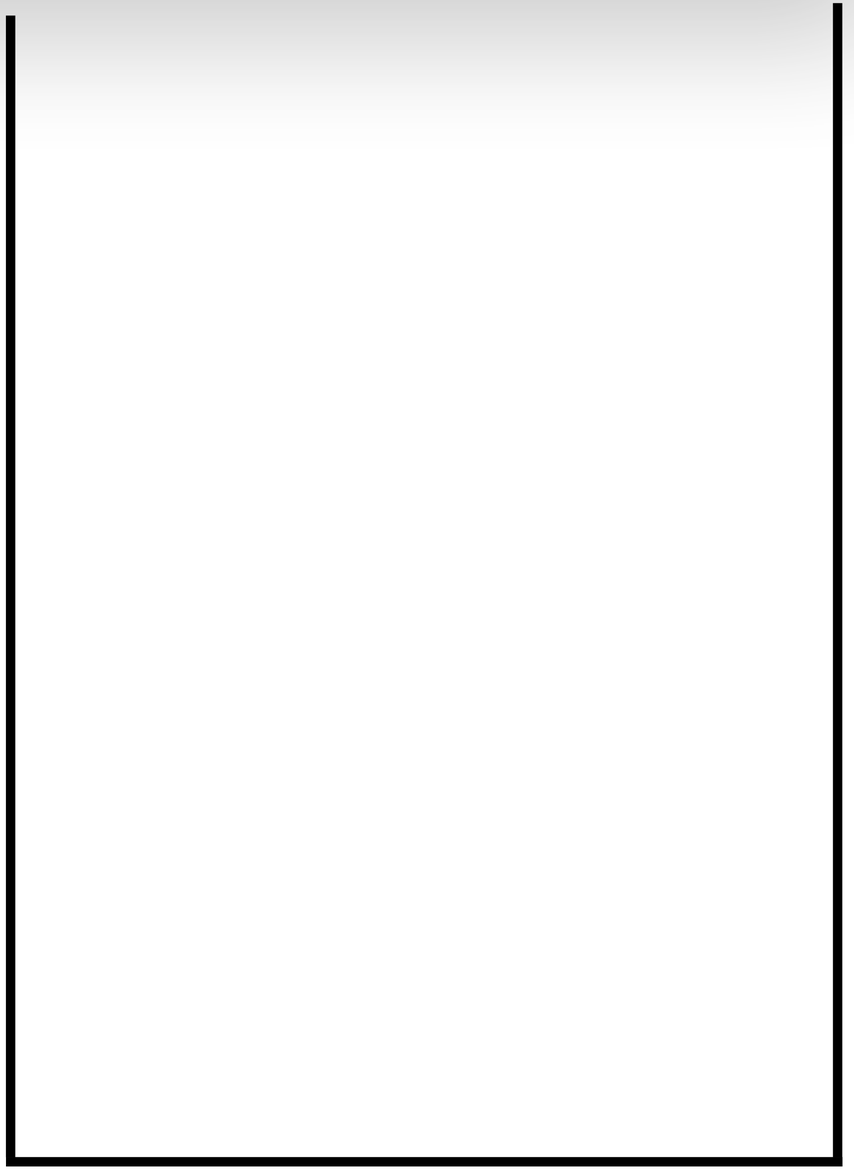


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

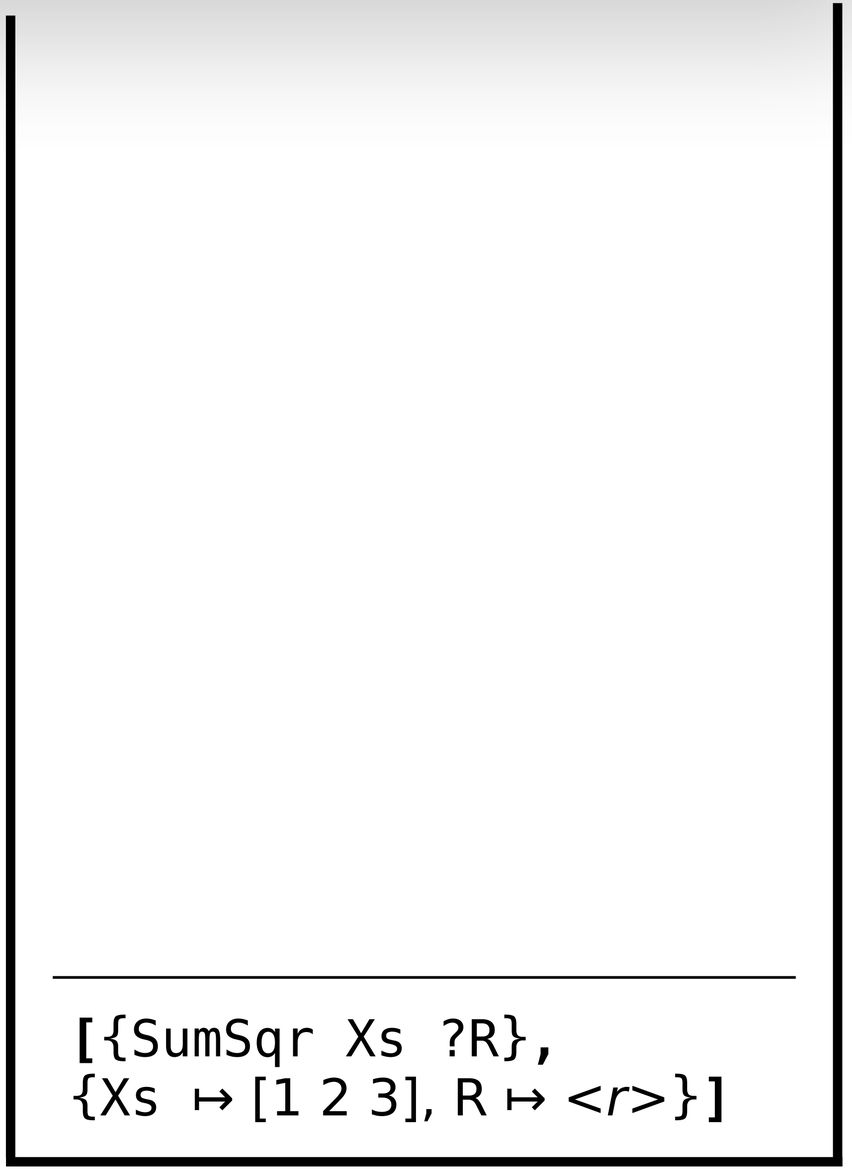


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

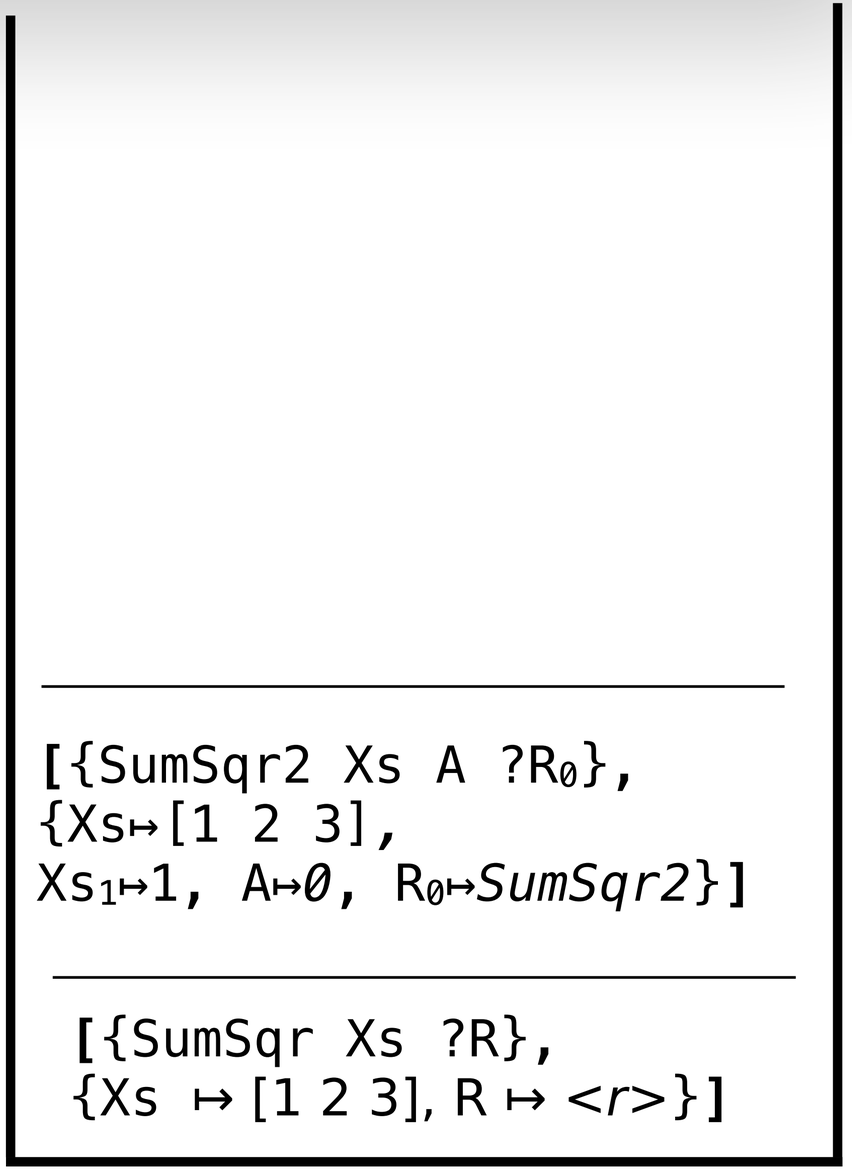


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

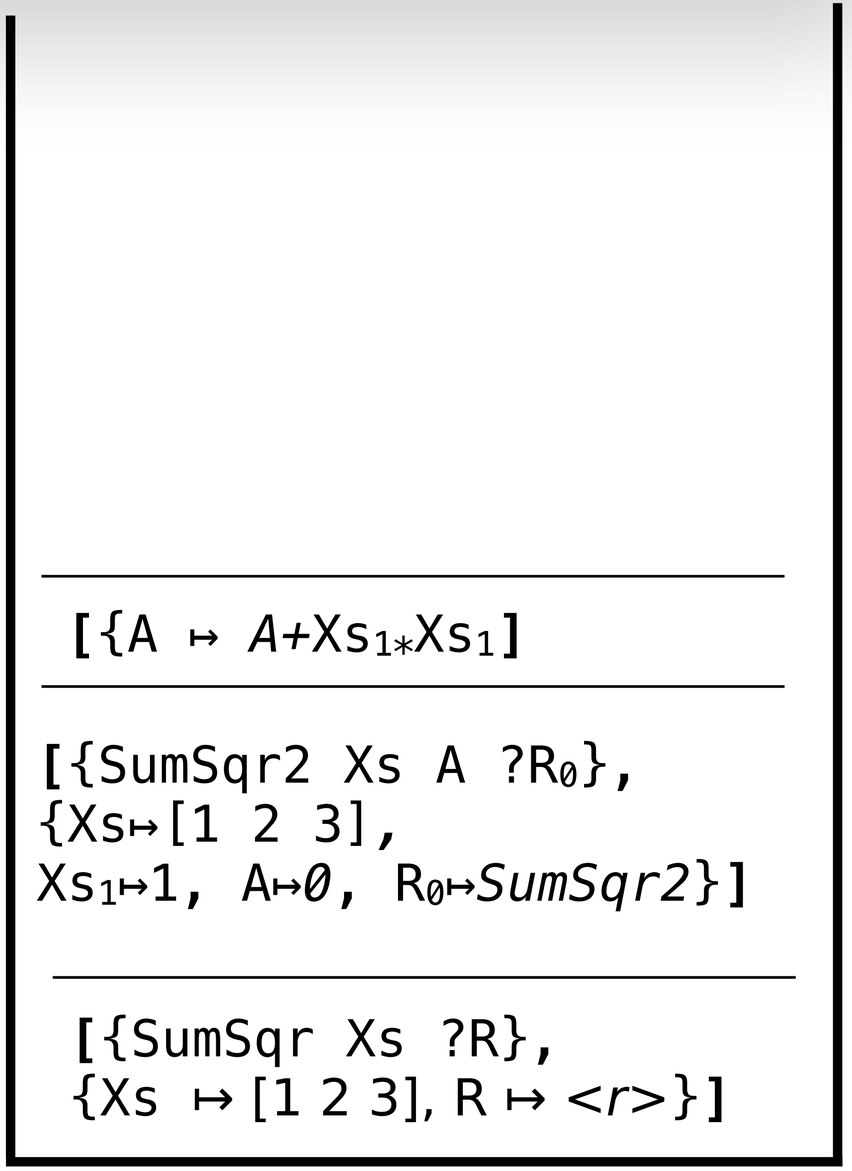


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

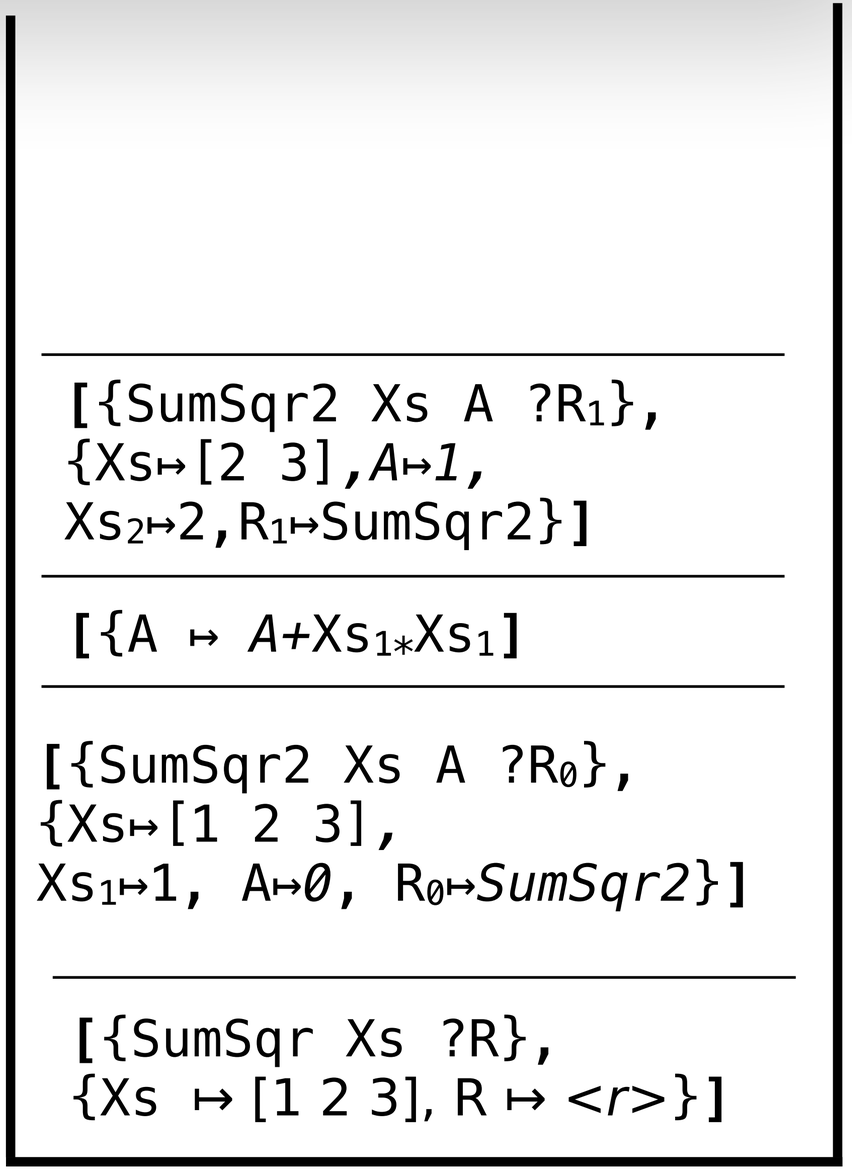


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

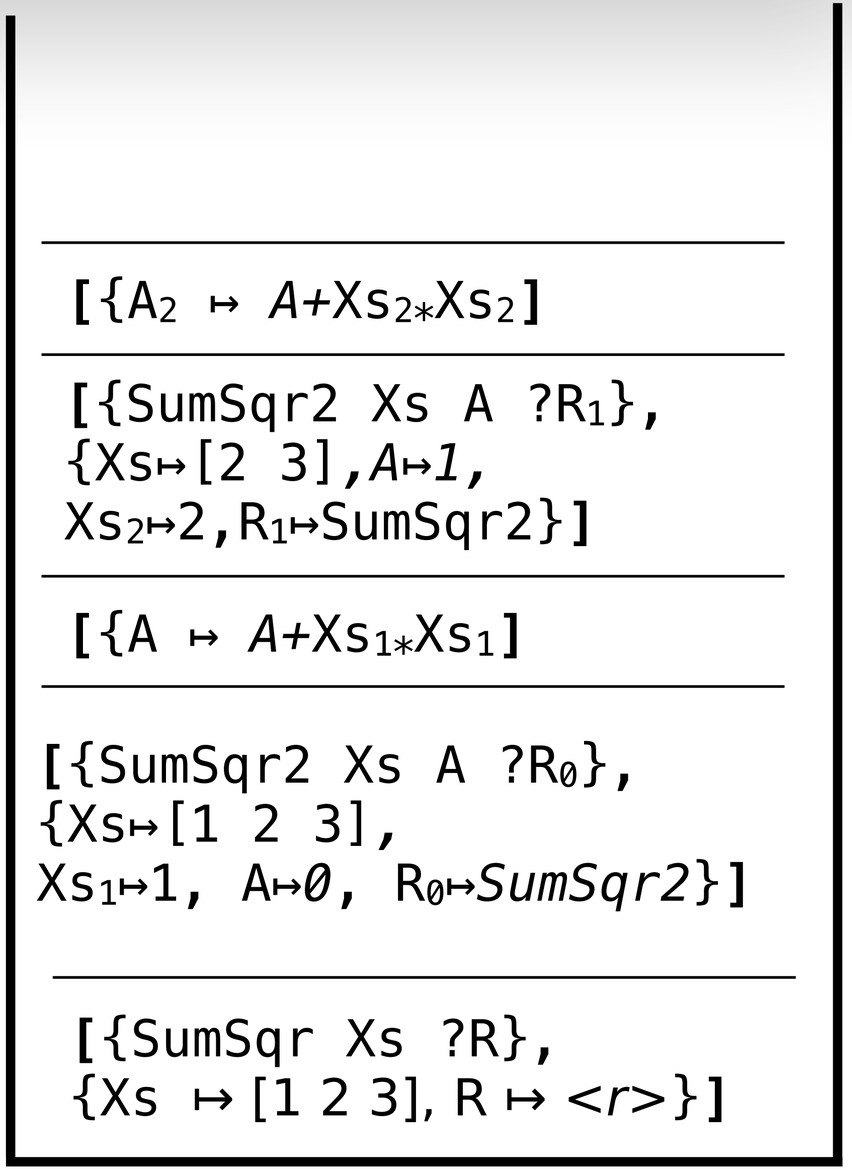


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Stack

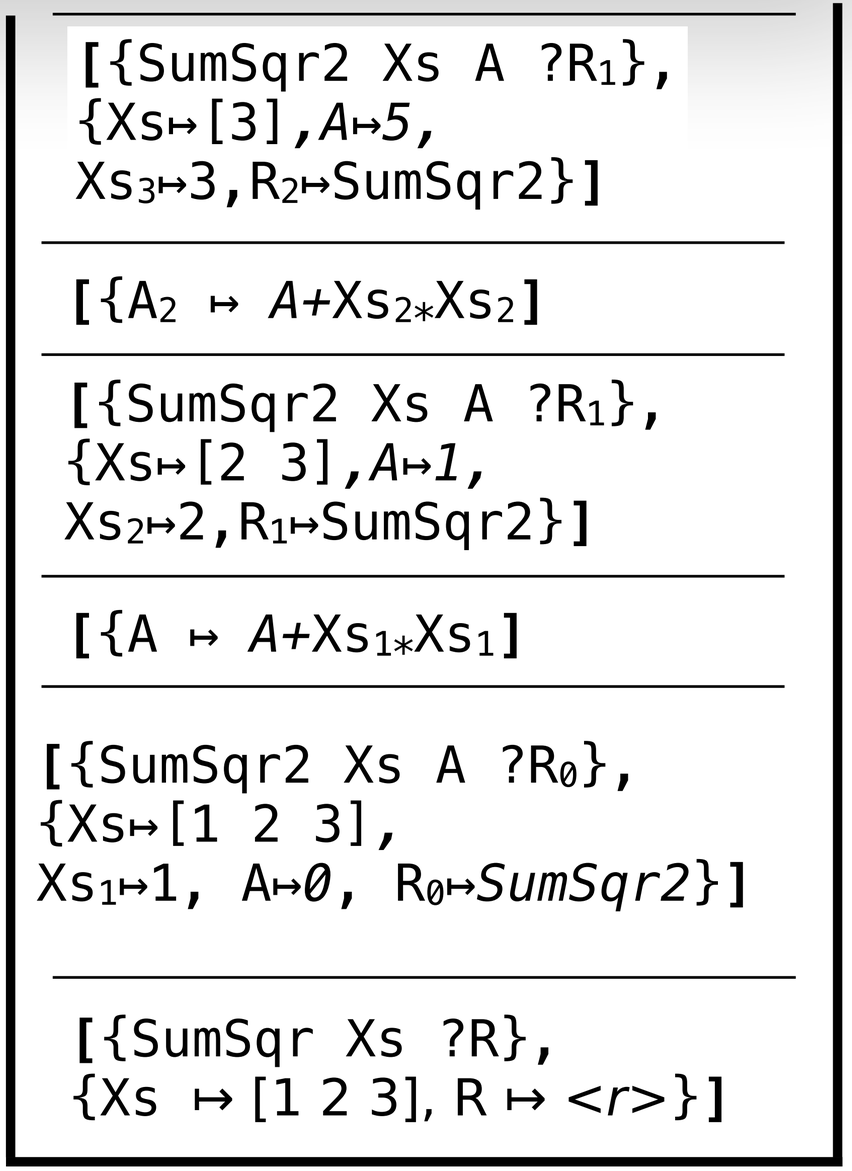


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

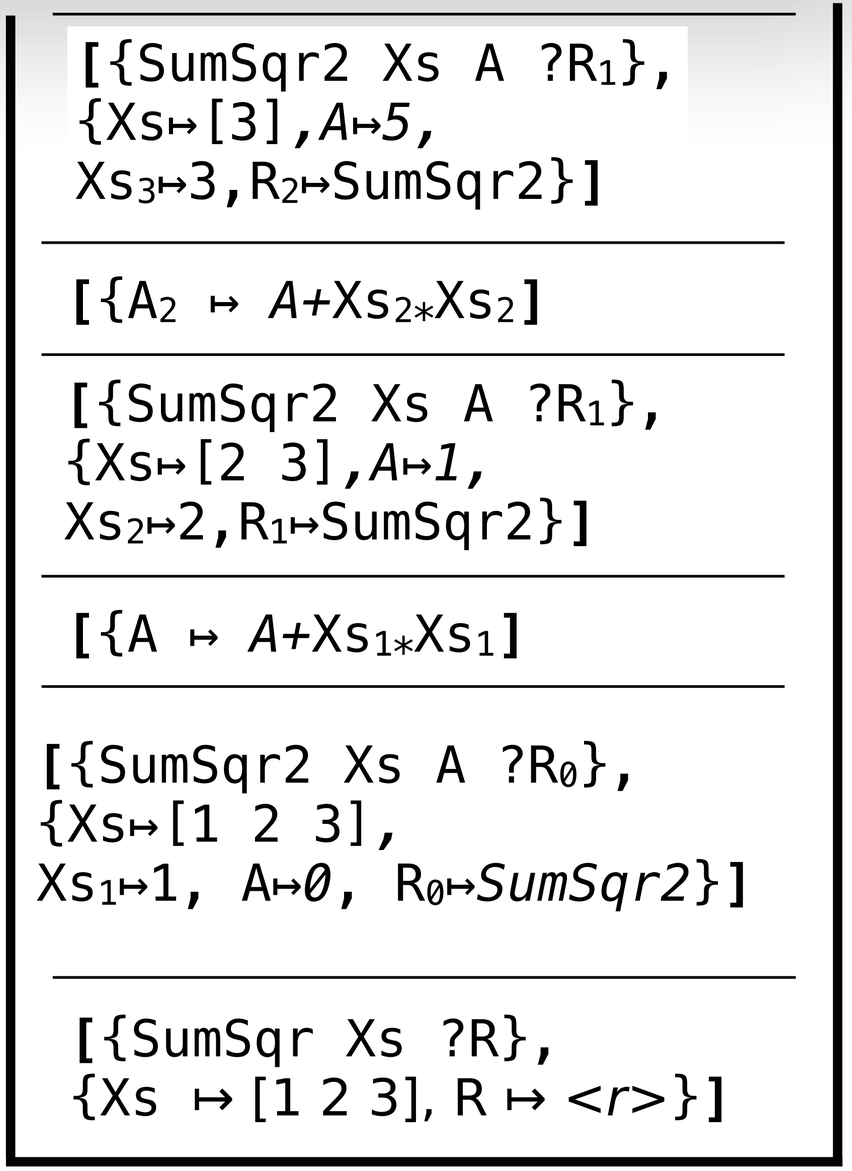
Stack



All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

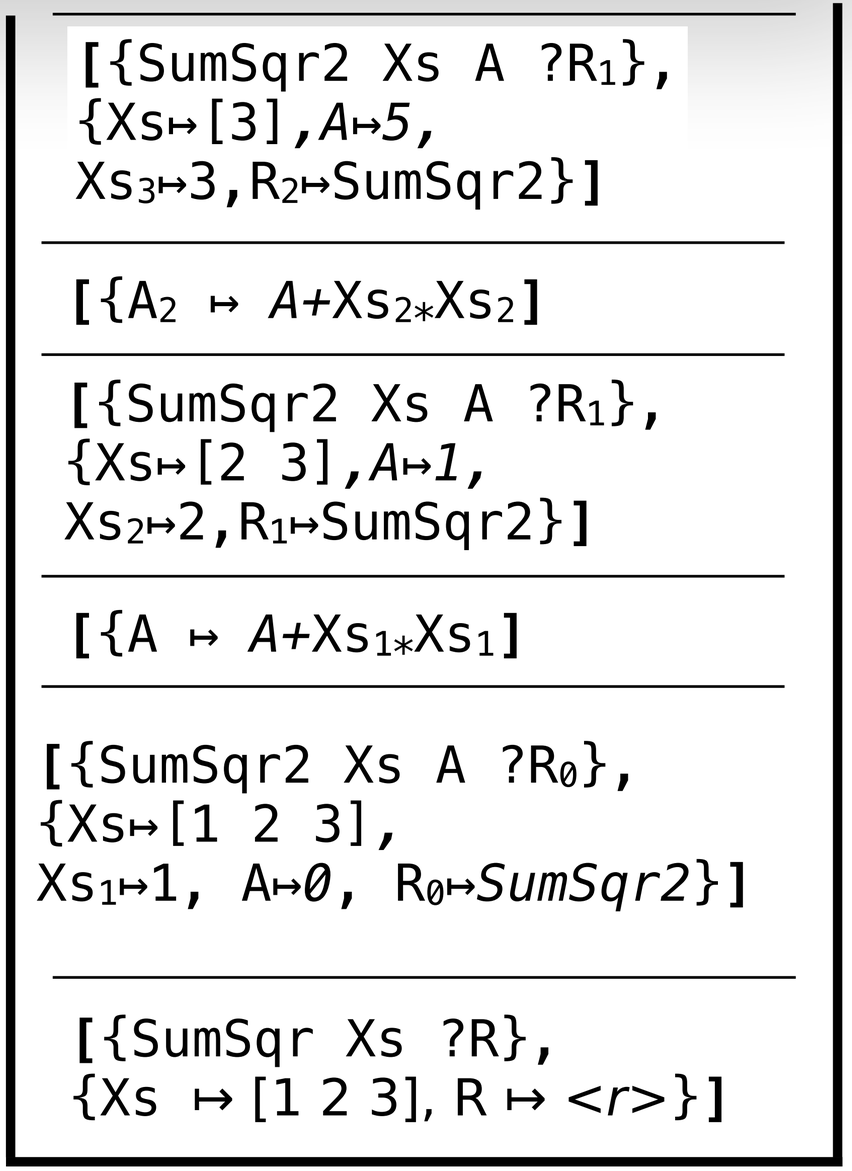


Stack

All about the stack

Solution 2

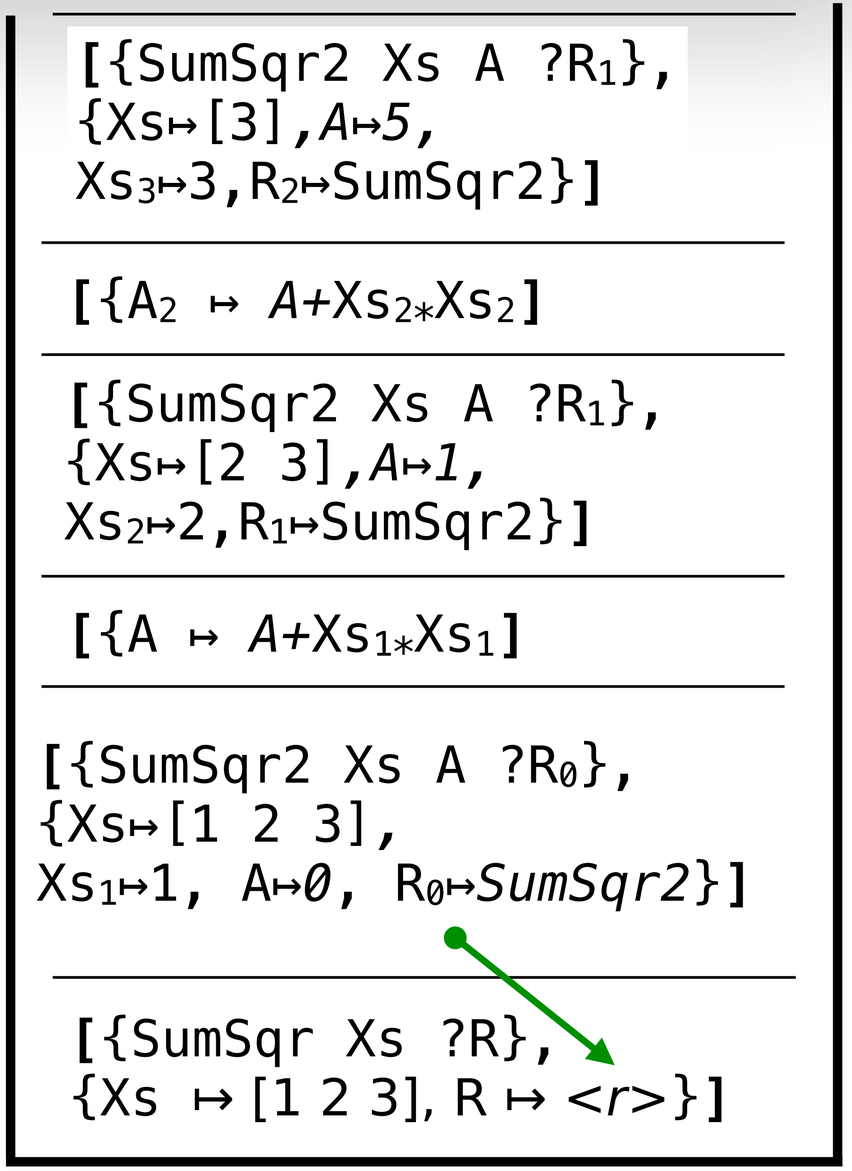
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

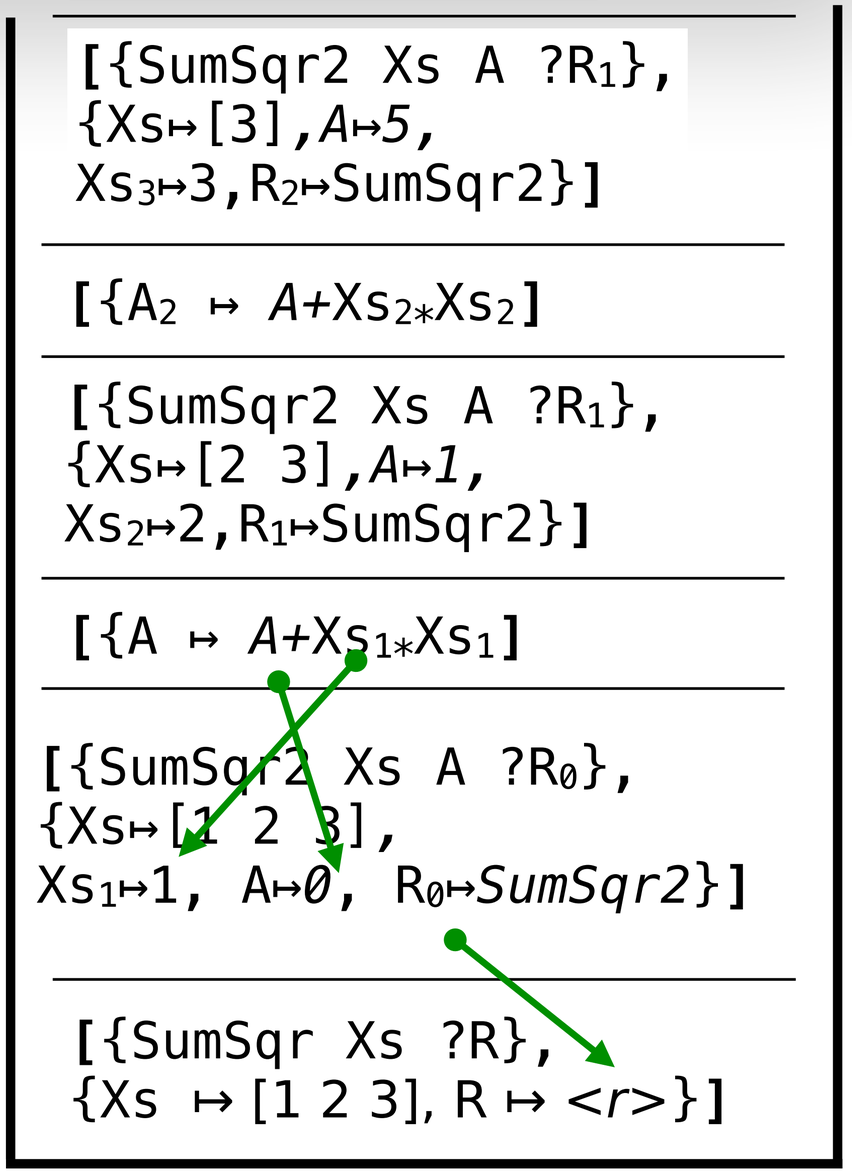
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

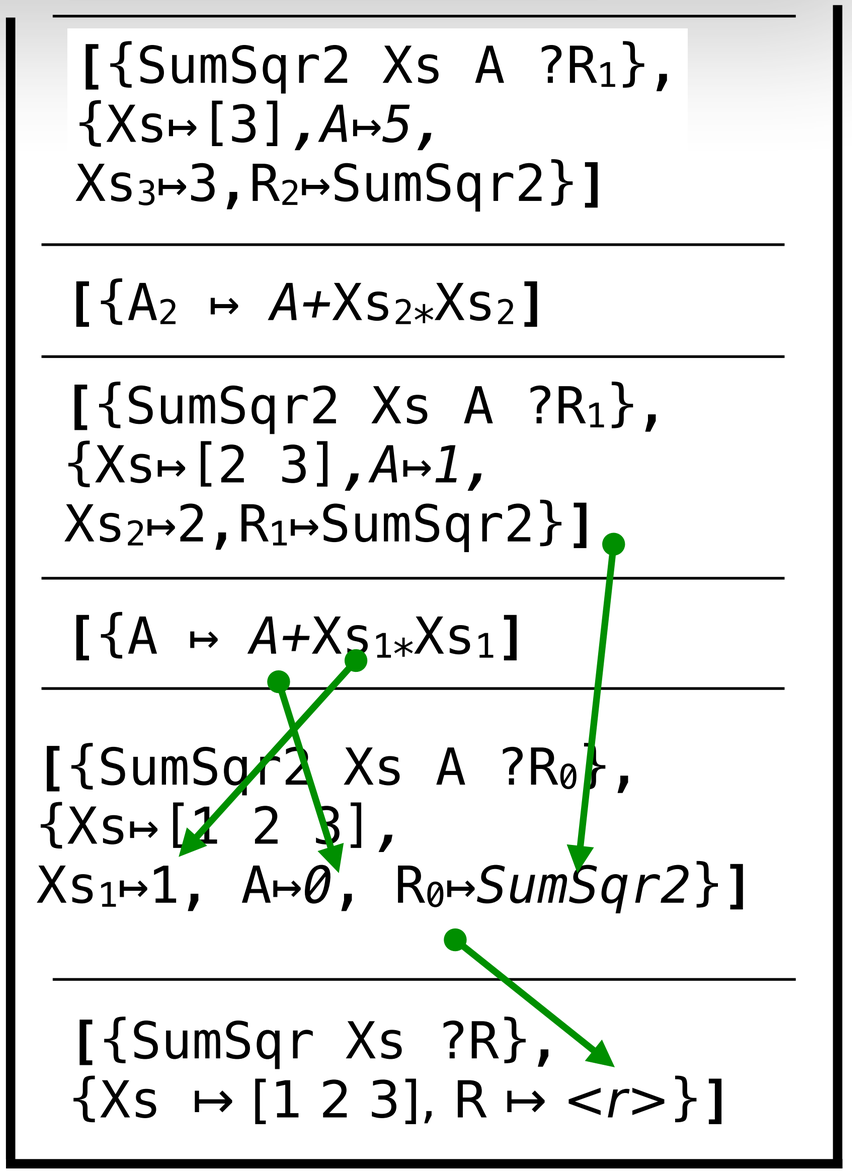


Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

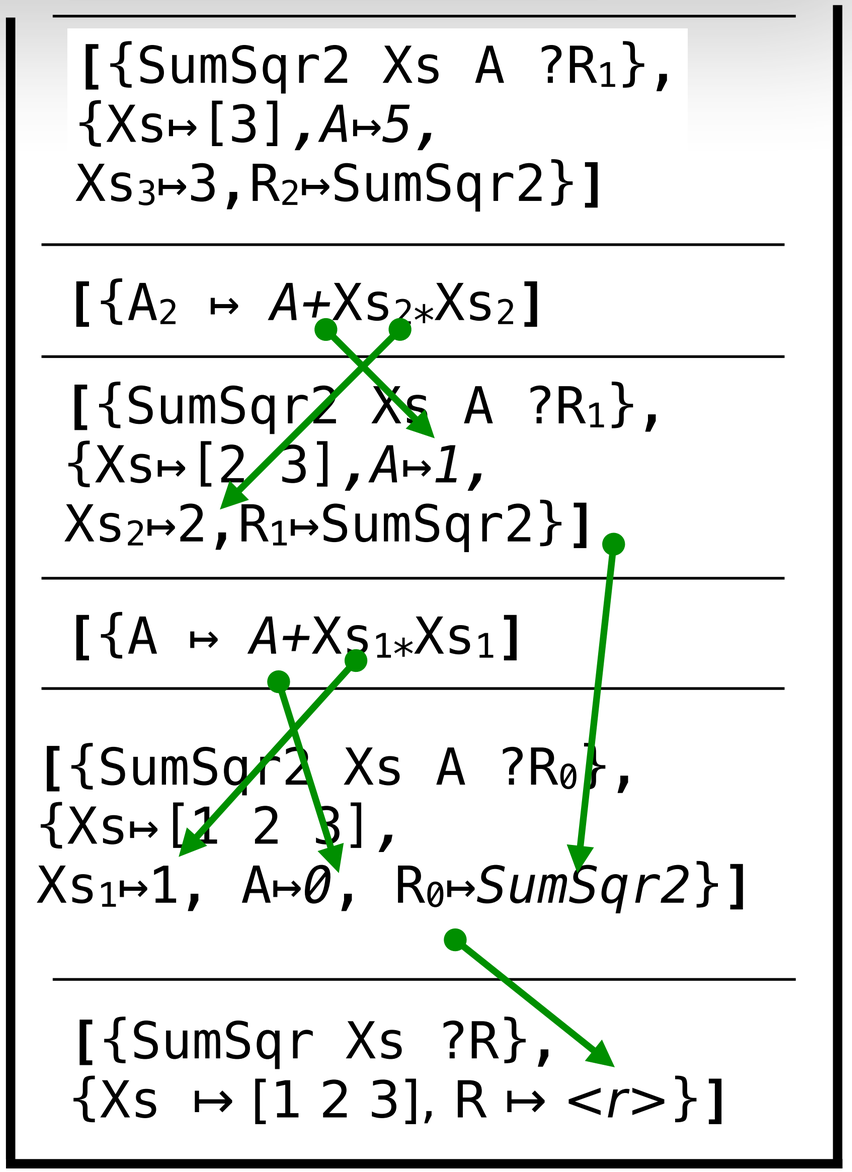


Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

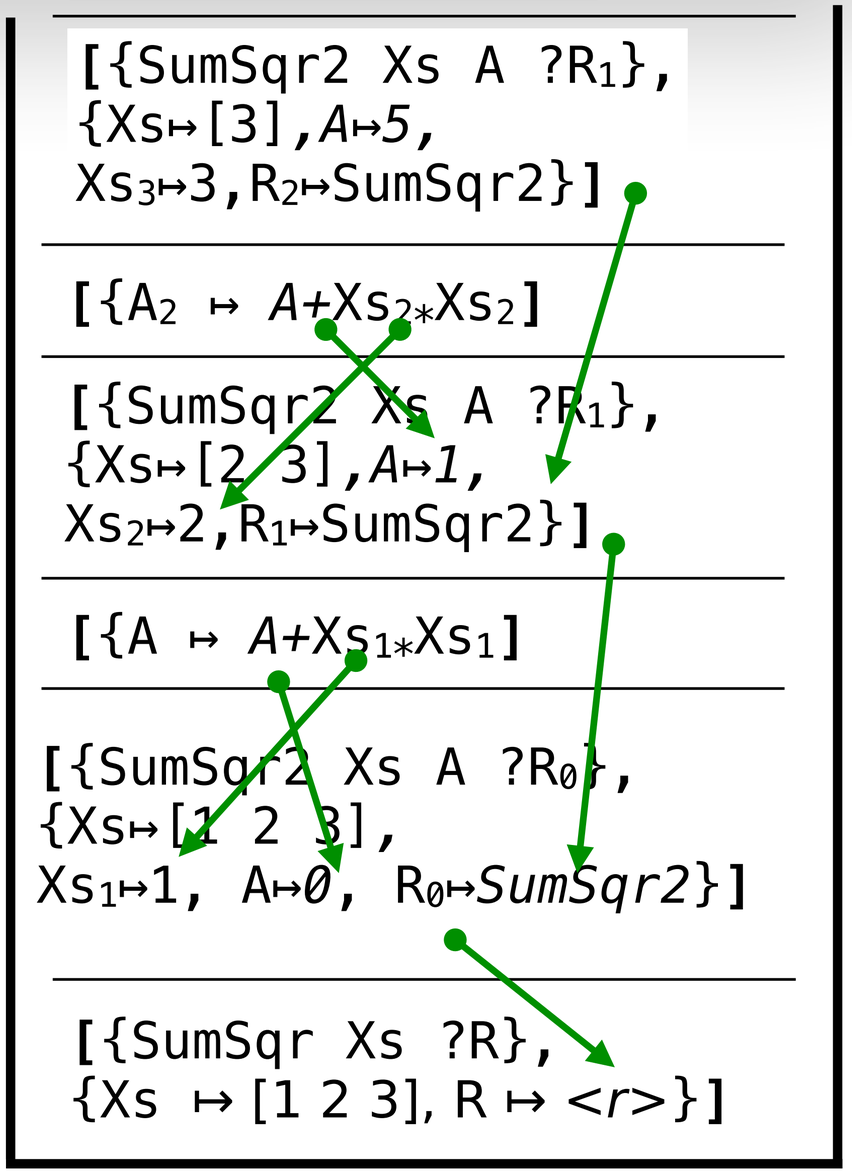


Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

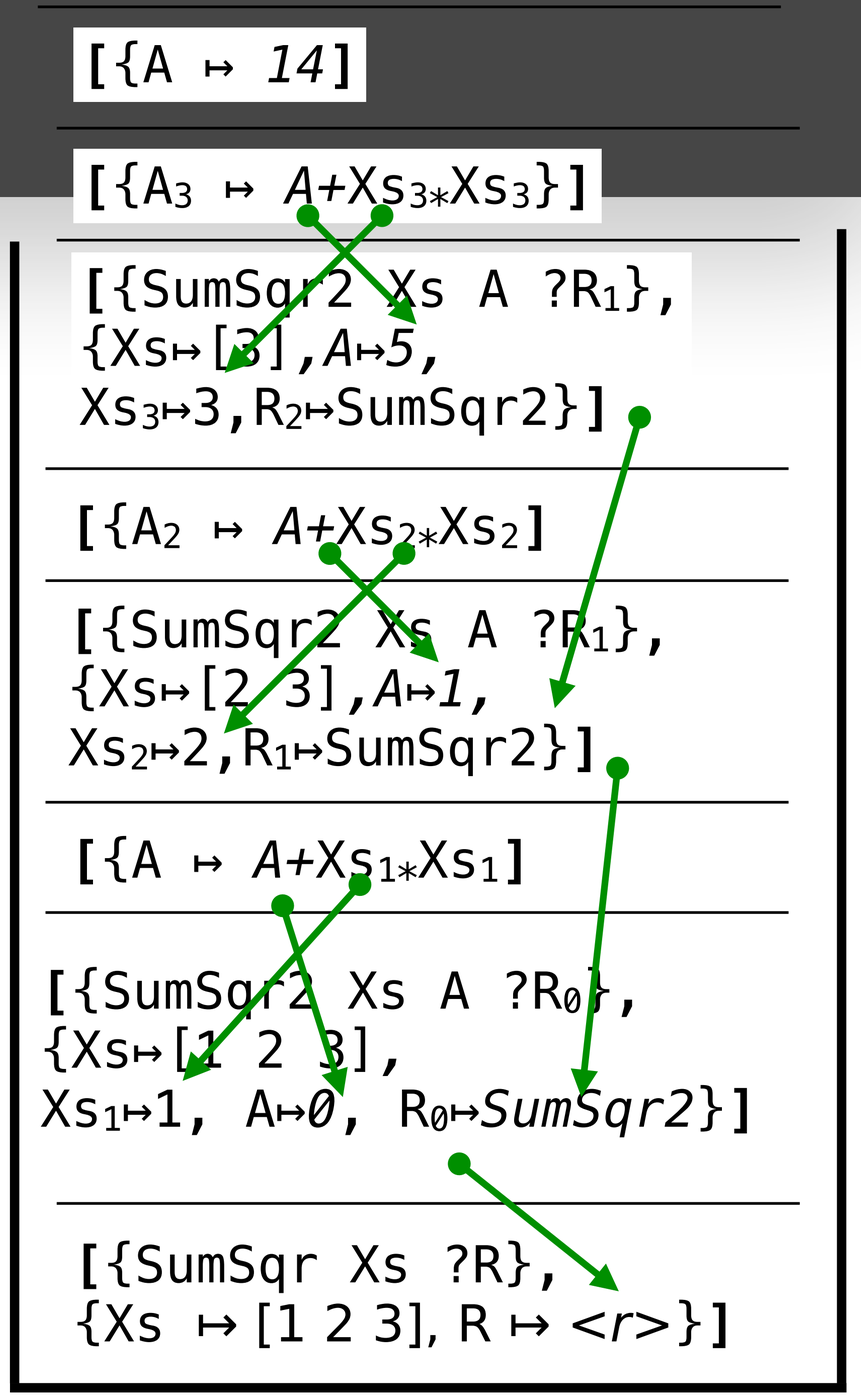


Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

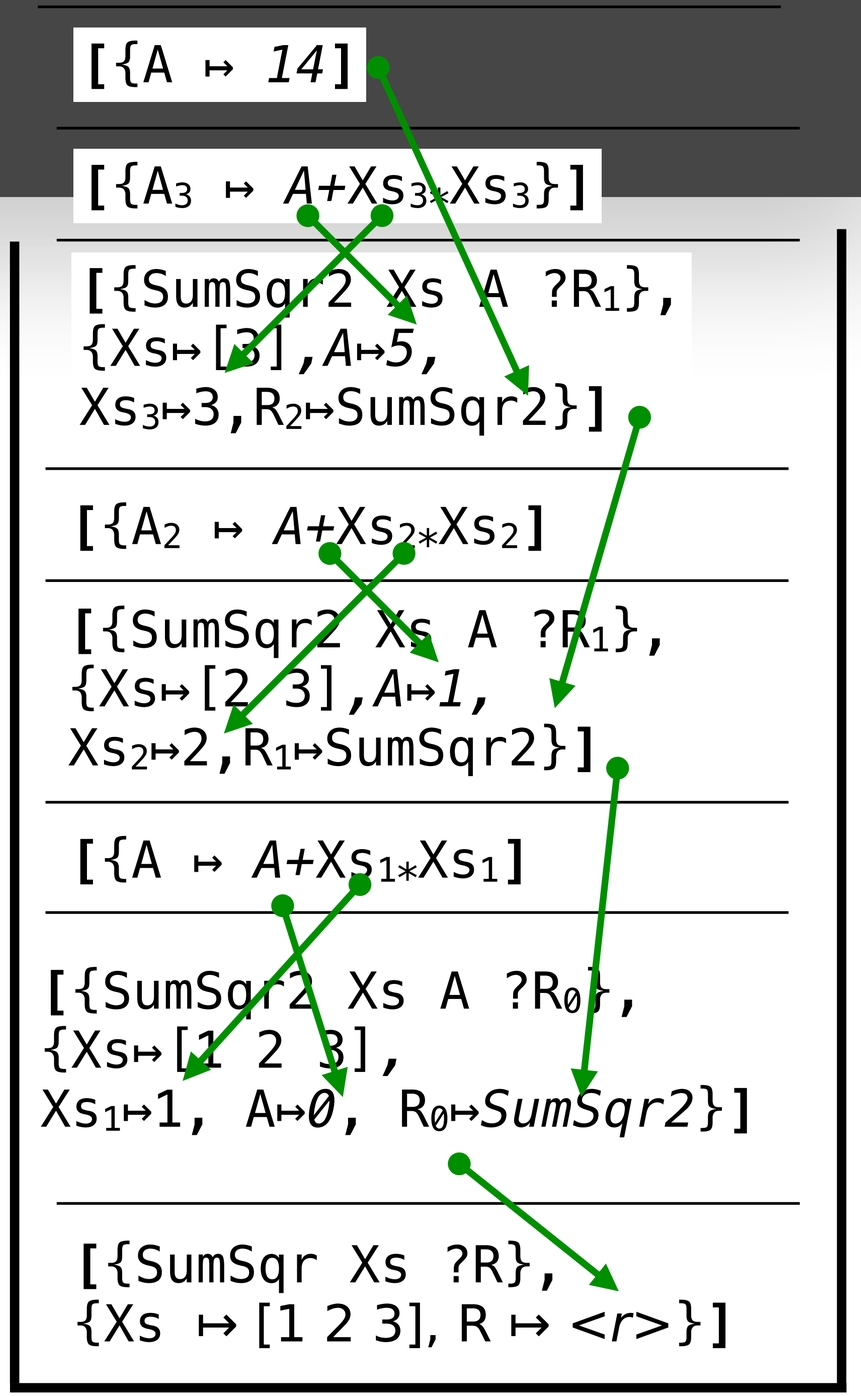


Stack

All about the stack

Solution 2

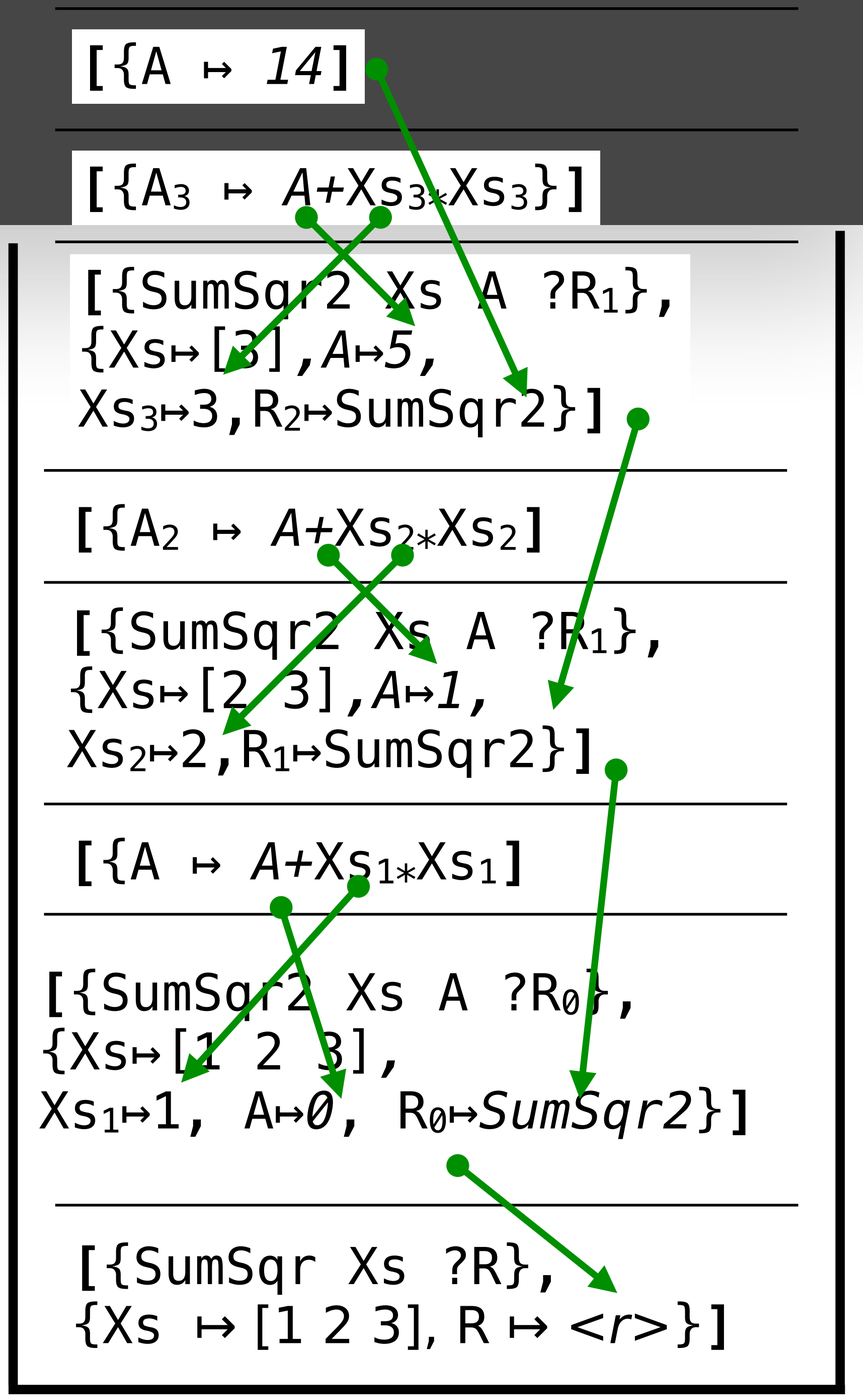
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

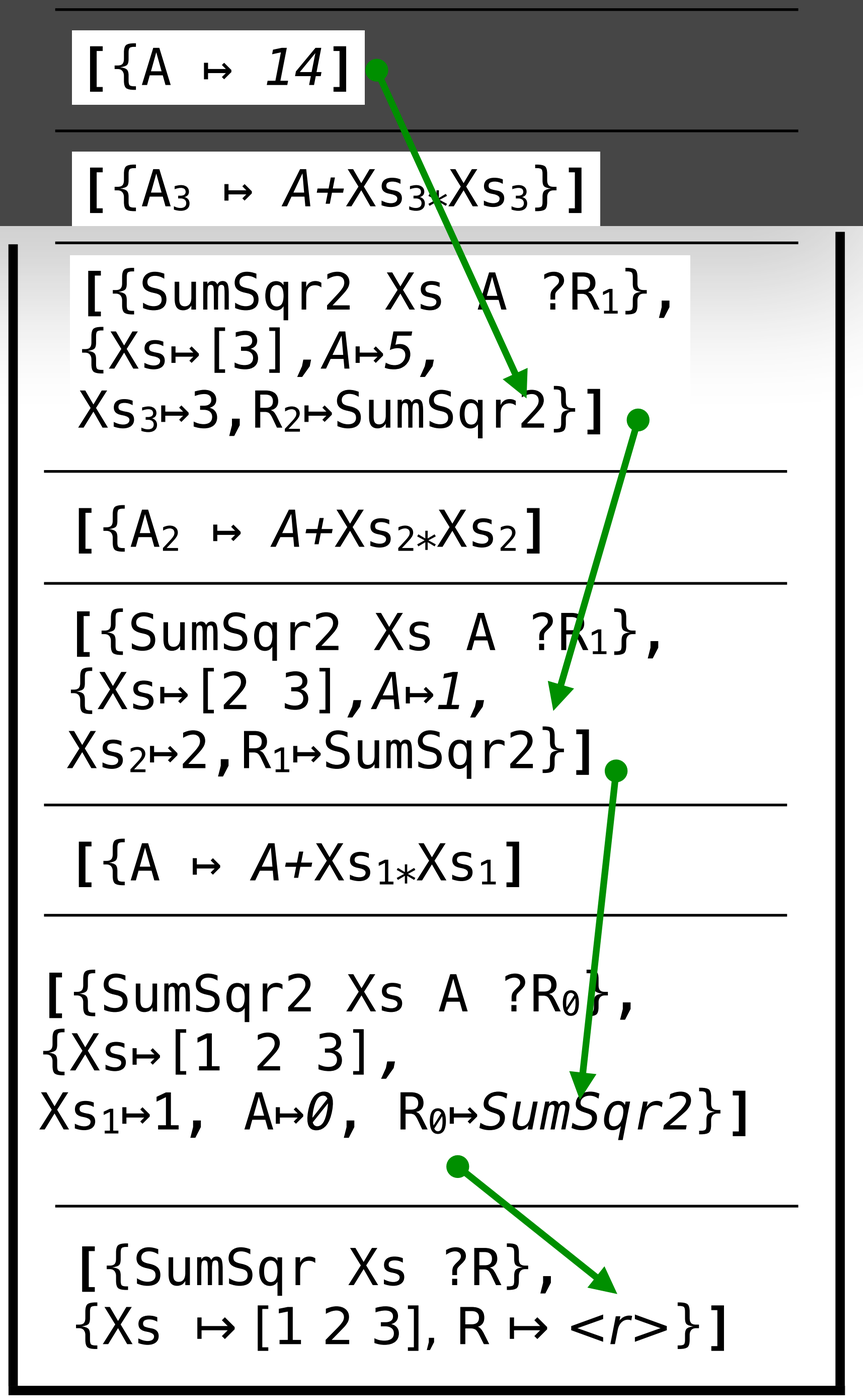


Stack

All about the stack

Solution 2

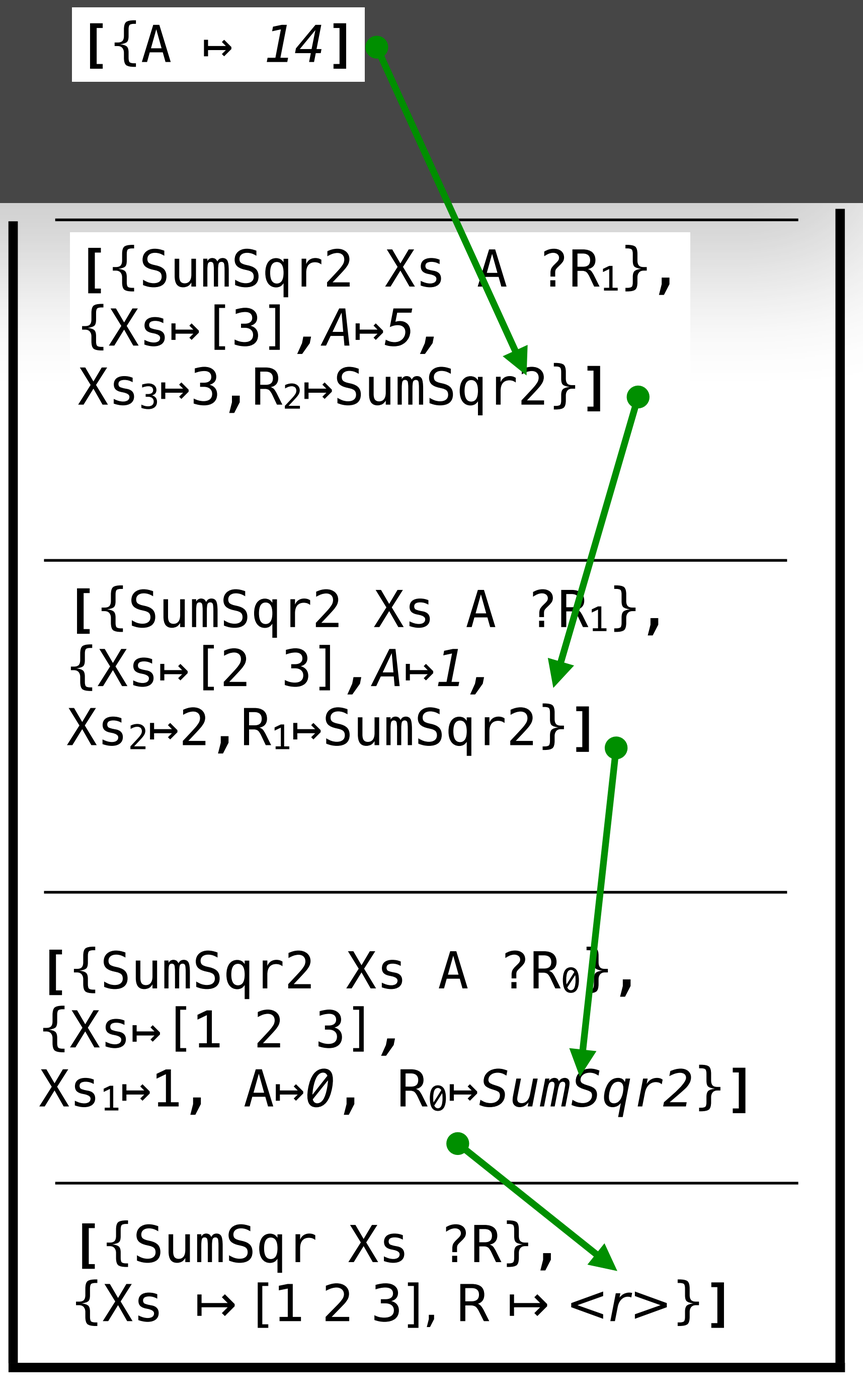
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

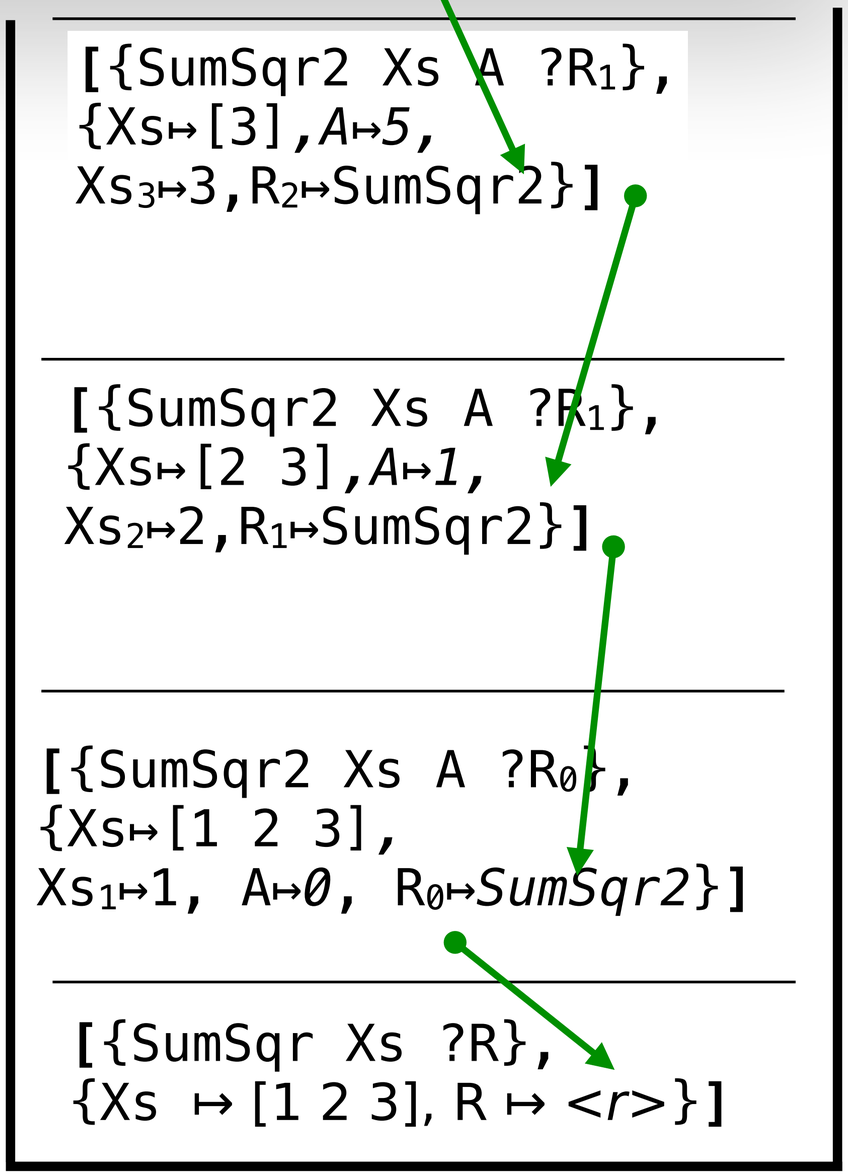
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

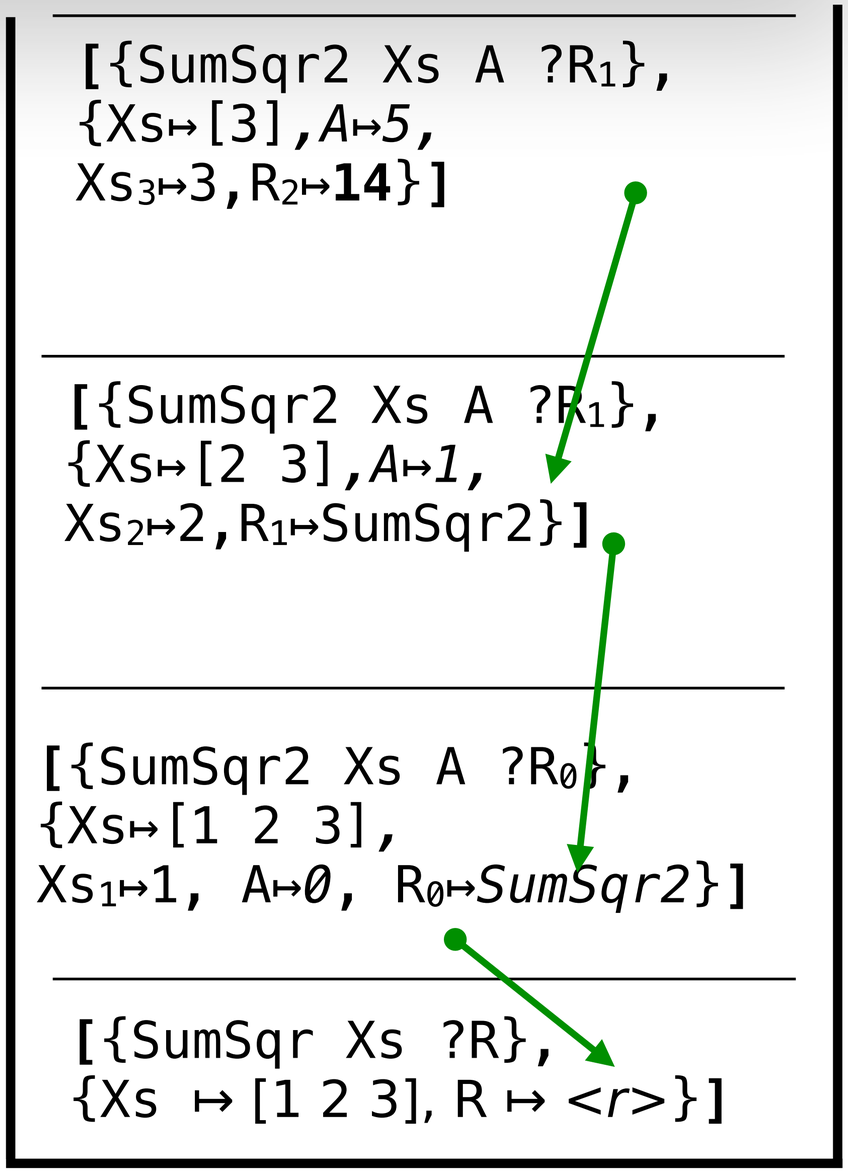
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
     {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

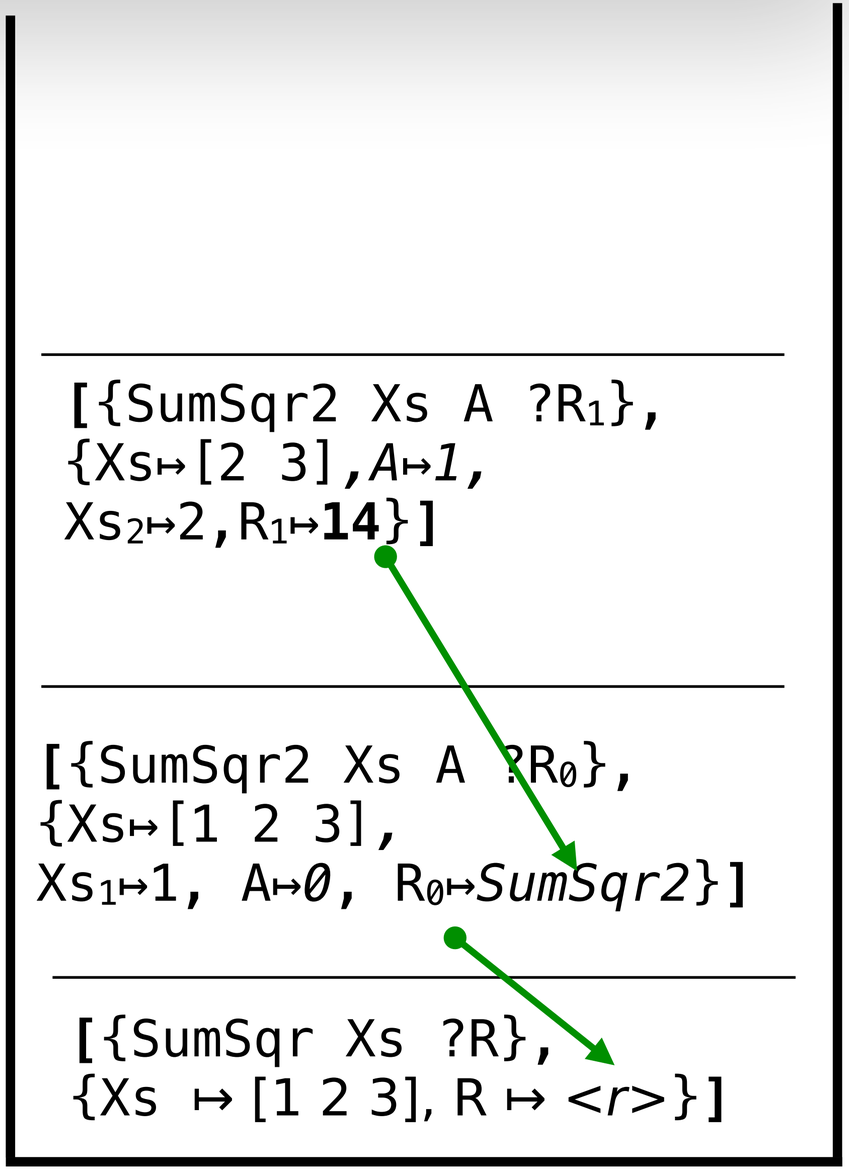
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

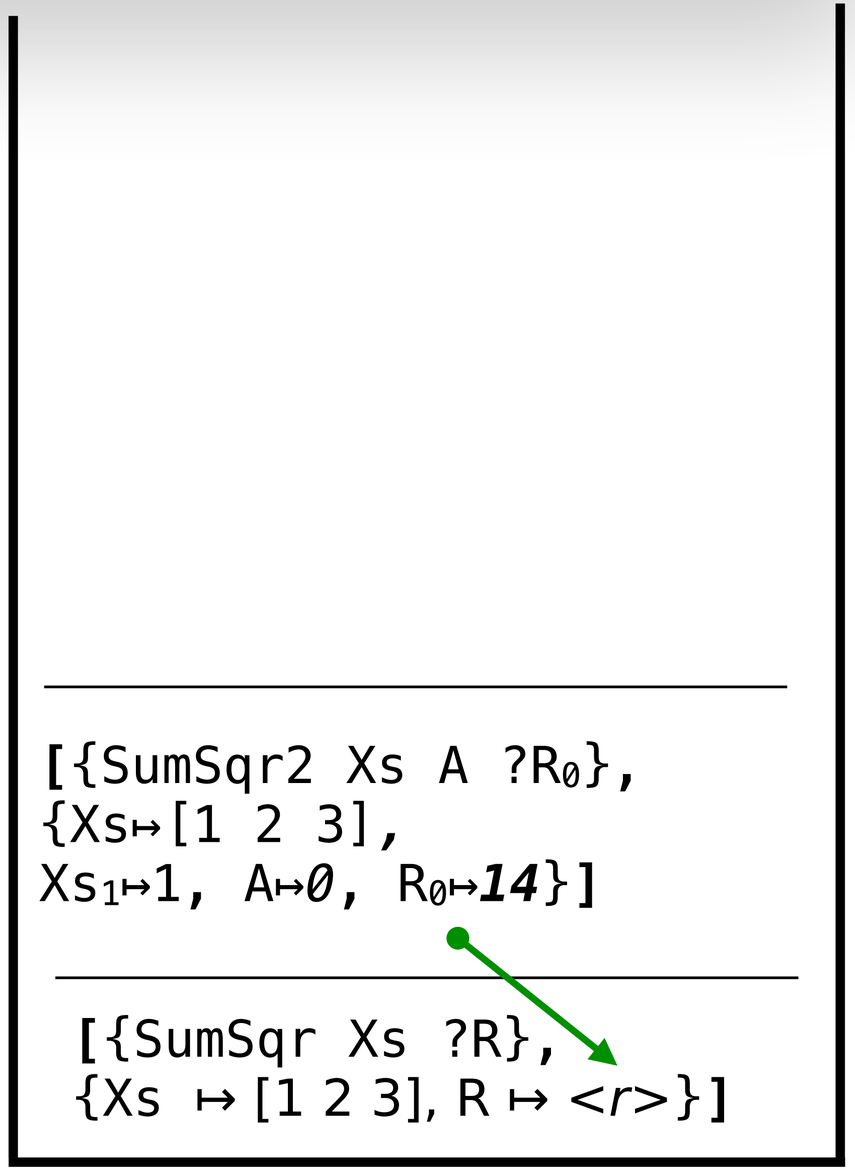
```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```



All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

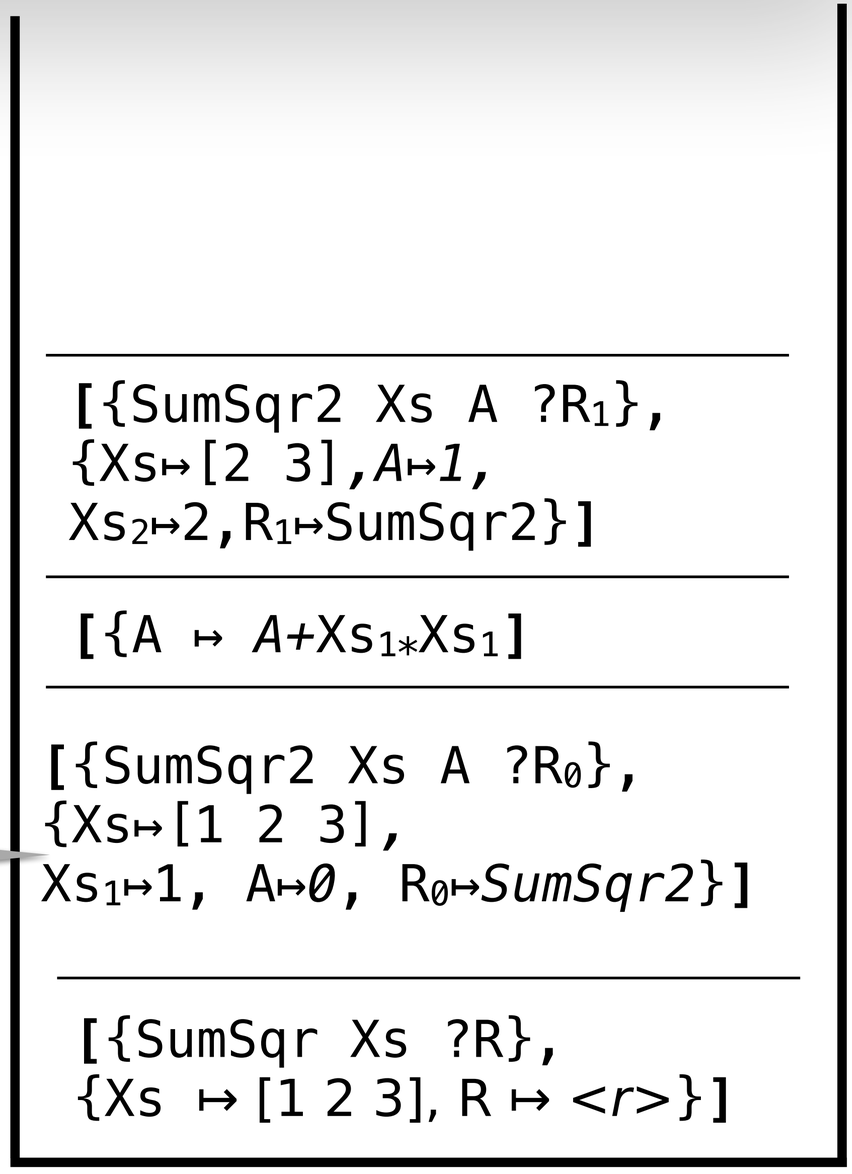


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame



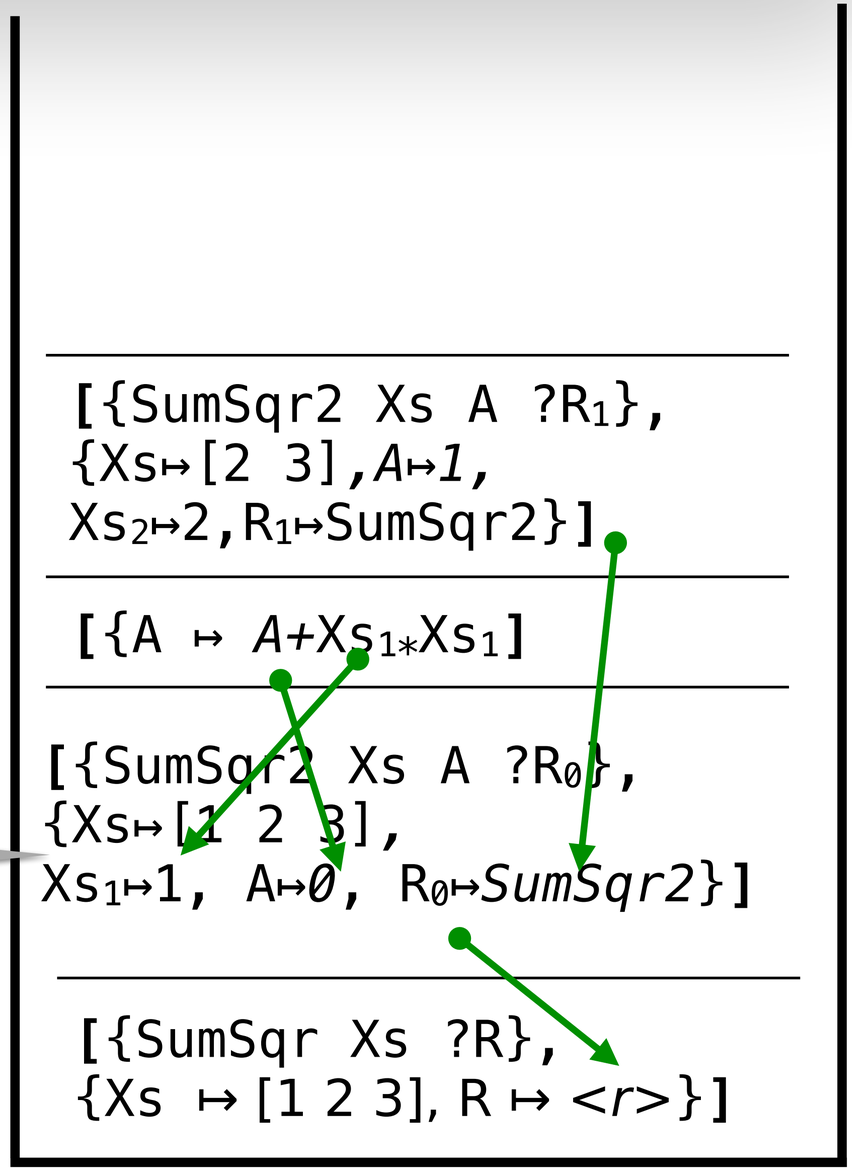
Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame



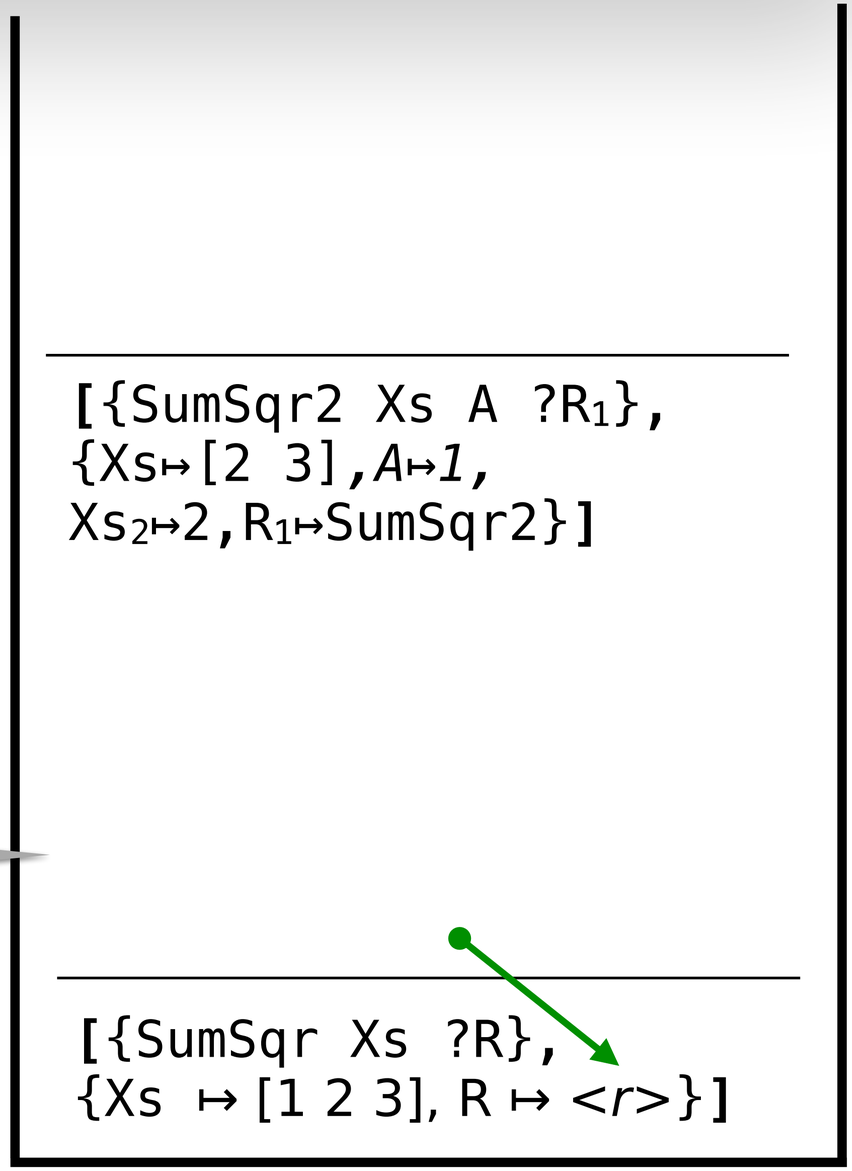
Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame

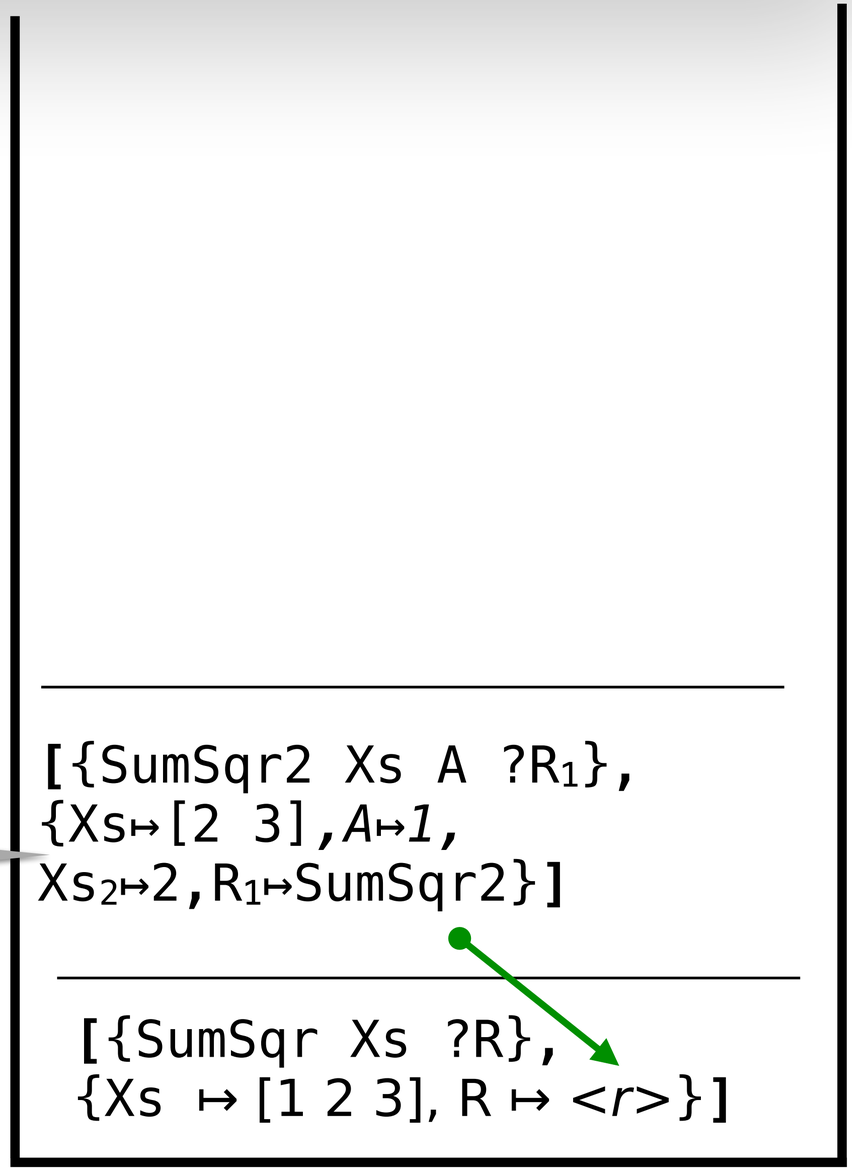


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame

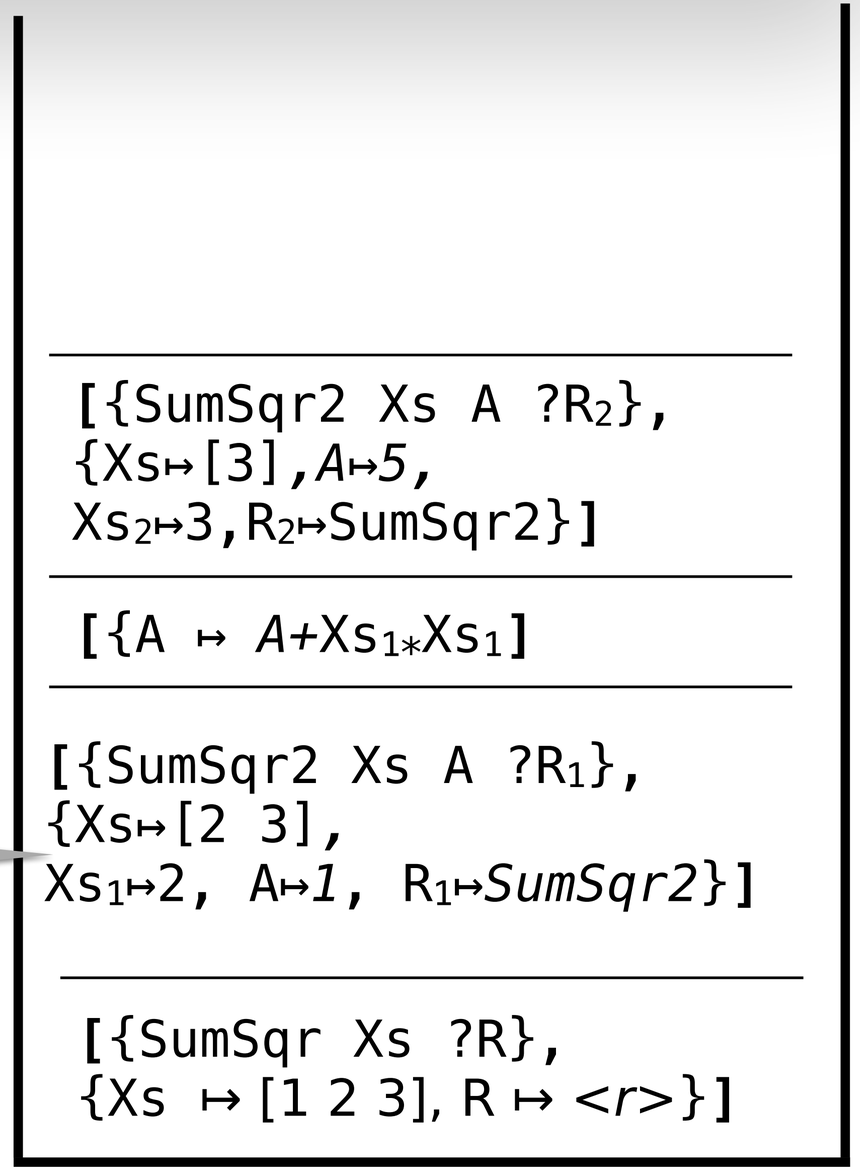


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame



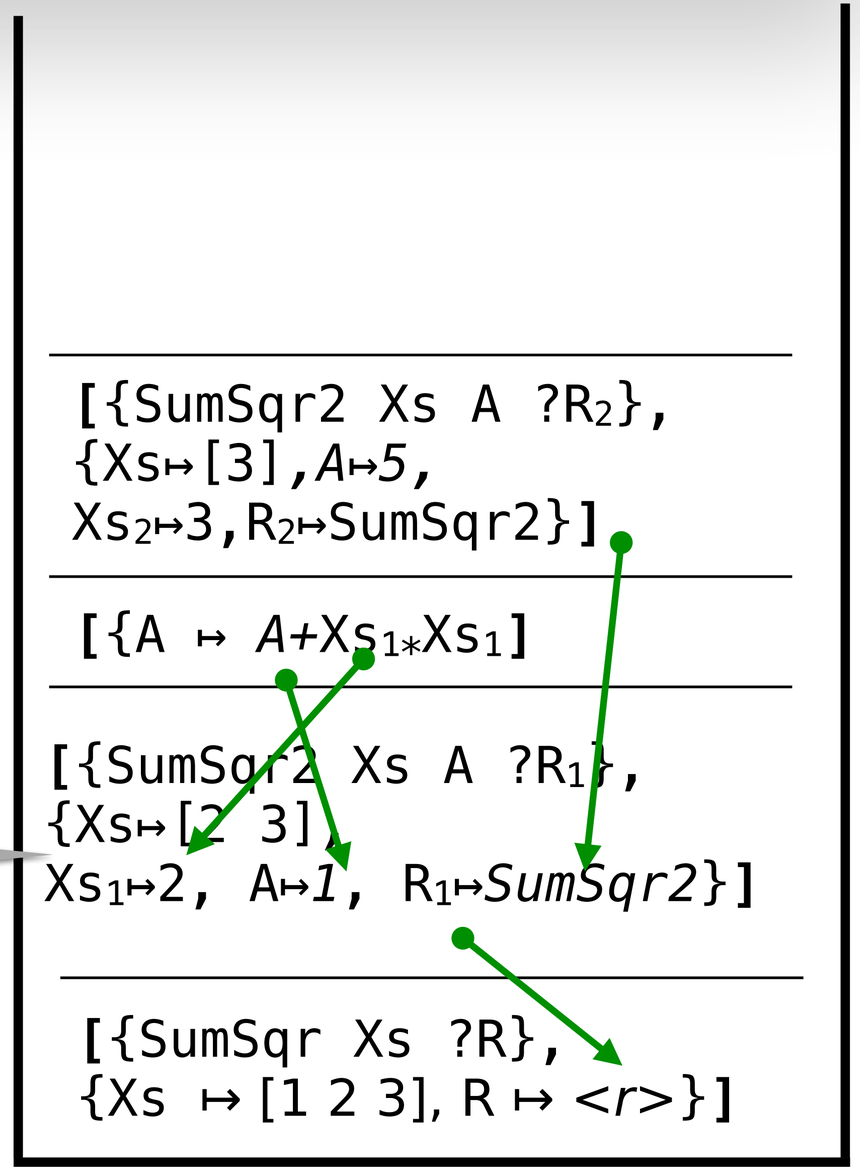
Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame



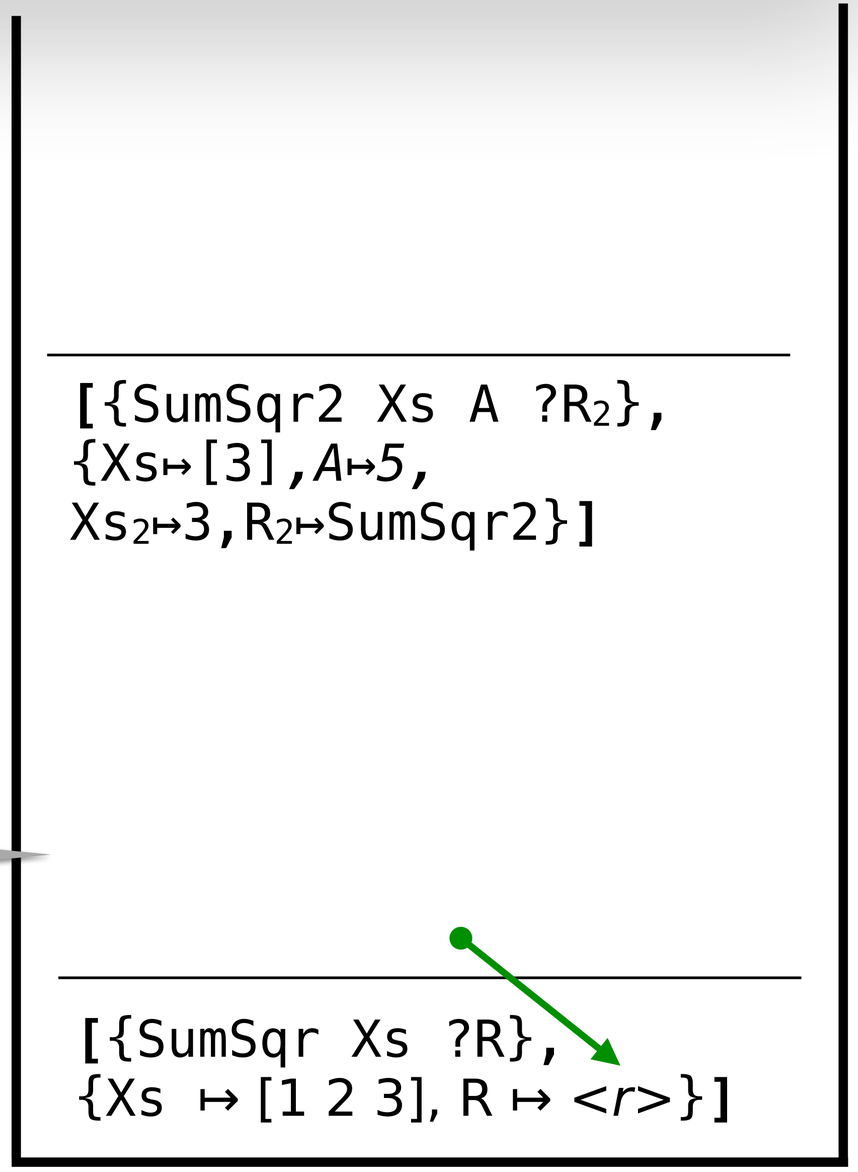
Stack

All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame

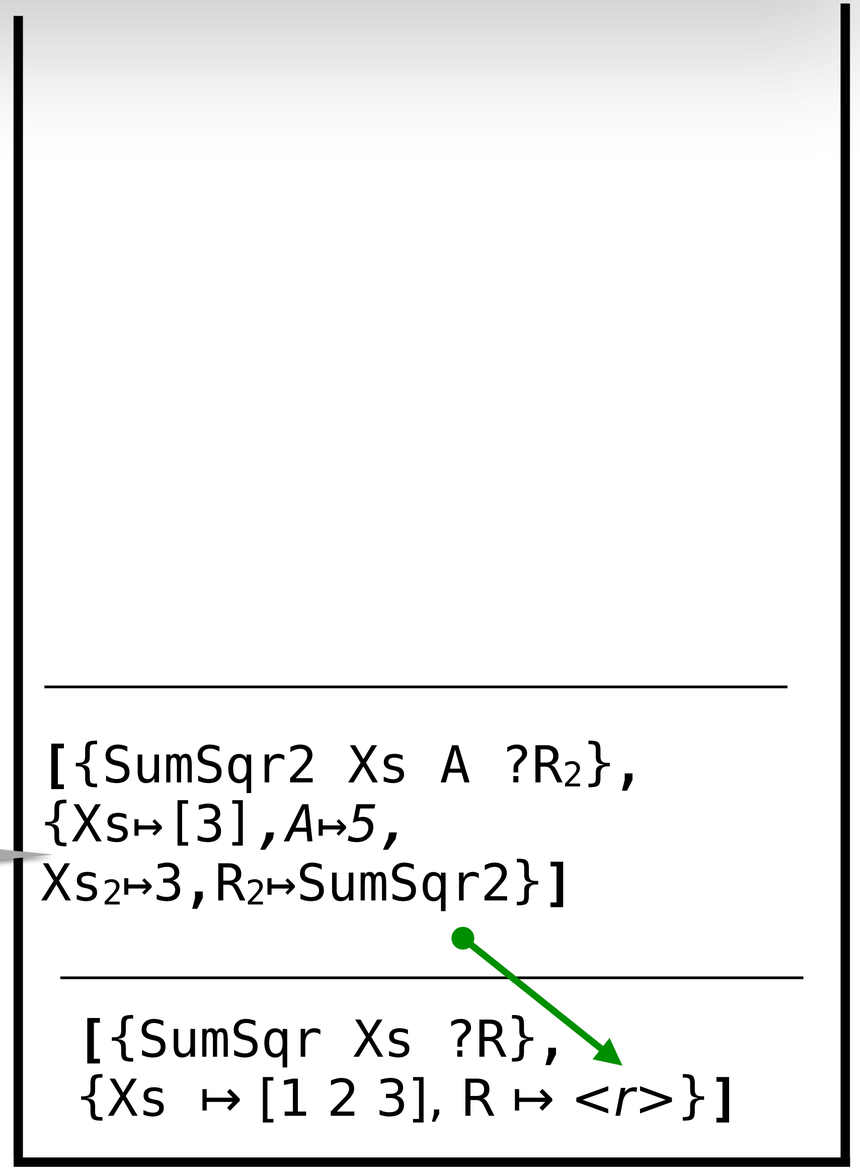


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame

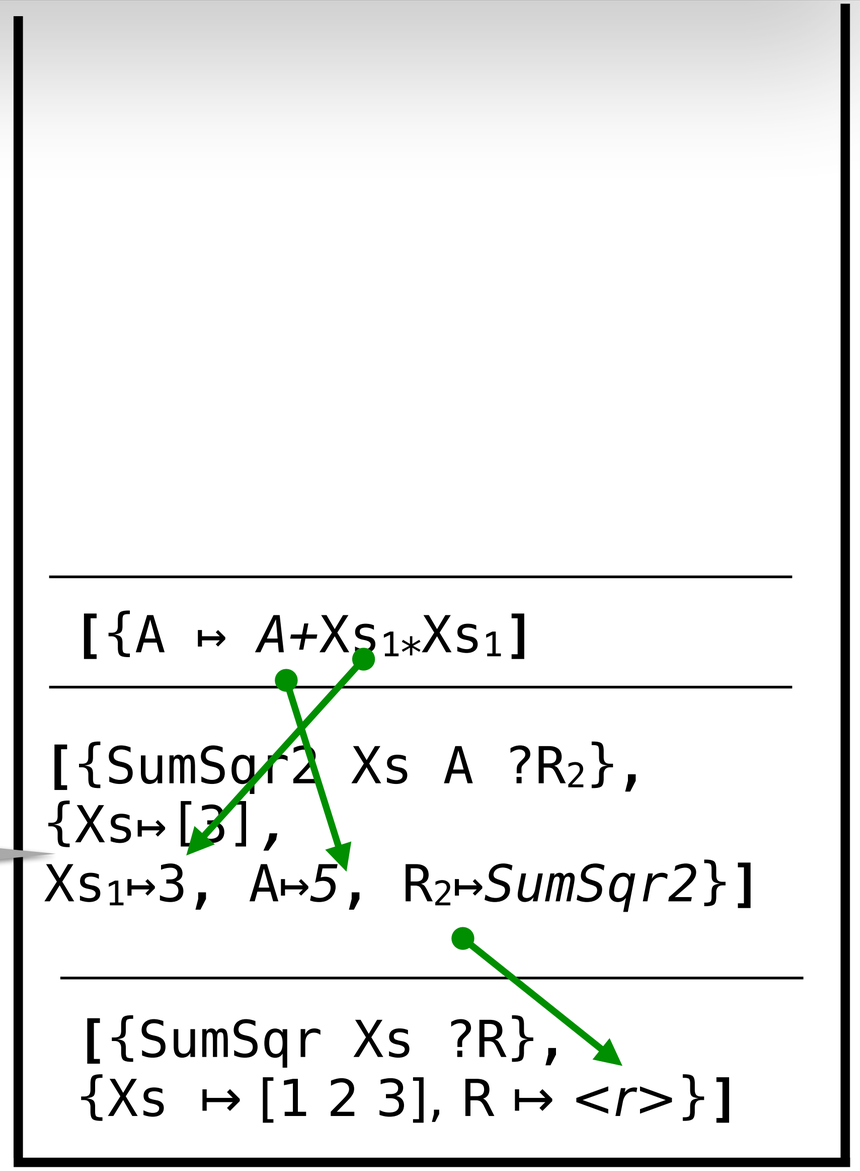


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame

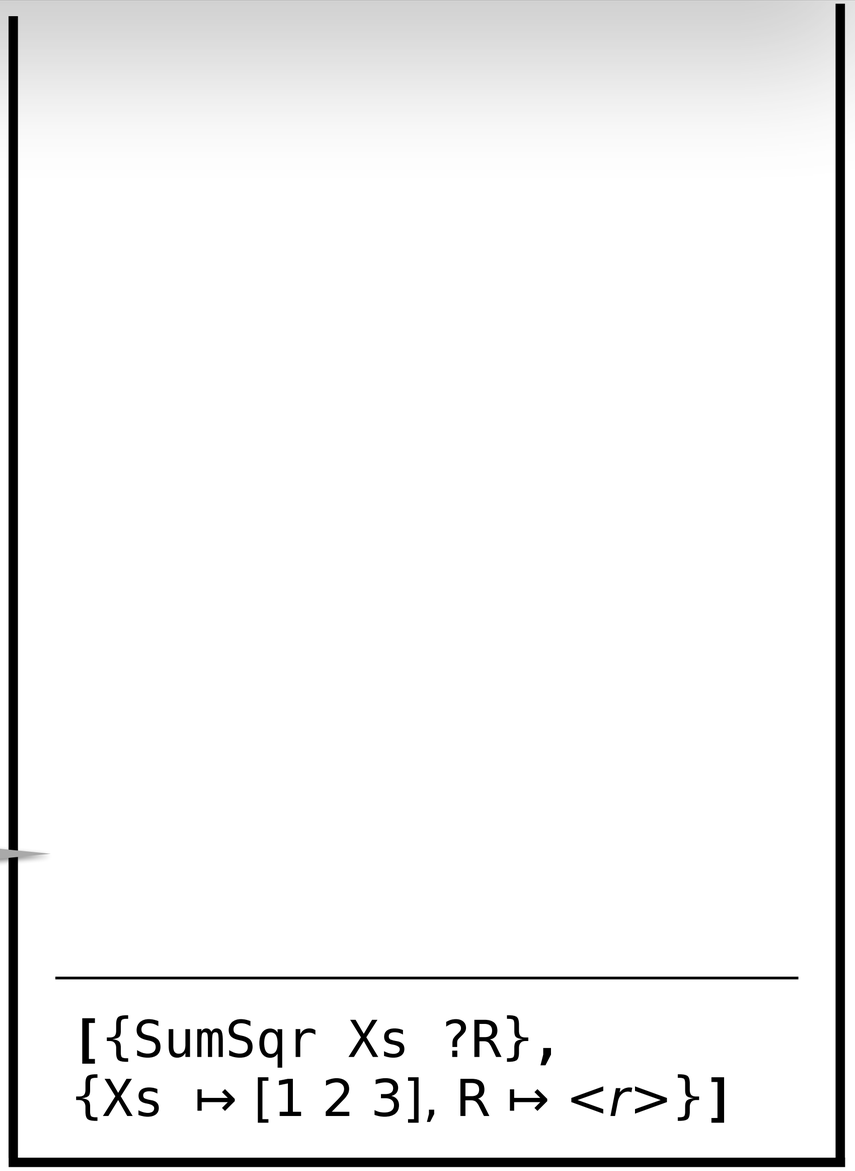


All about the stack

Solution 2

```
fun {SumSqr Lst}
 local SumSqr2 in
  fun {SumSqr2 Lst Accum}
   case Lst of nil then Accum
    [] Car|Cdr then
      {SumSqr2 Cdr Accum+Car*Car}
    end
  end
 {SumSqrt2 Lst 0}
 end
end
```

Everything is resolved here, so I don't need this frame



TOTAL NUMBER OF
FRAMES THAT I NEED



Recursion

Recursive definitions are the definitions that **recursively** call themselves, like this one

- Structures
- Procedures

Procedures with a **constant stack** are call **iterative**

Procedures with a **growing stack** are call **recursive**

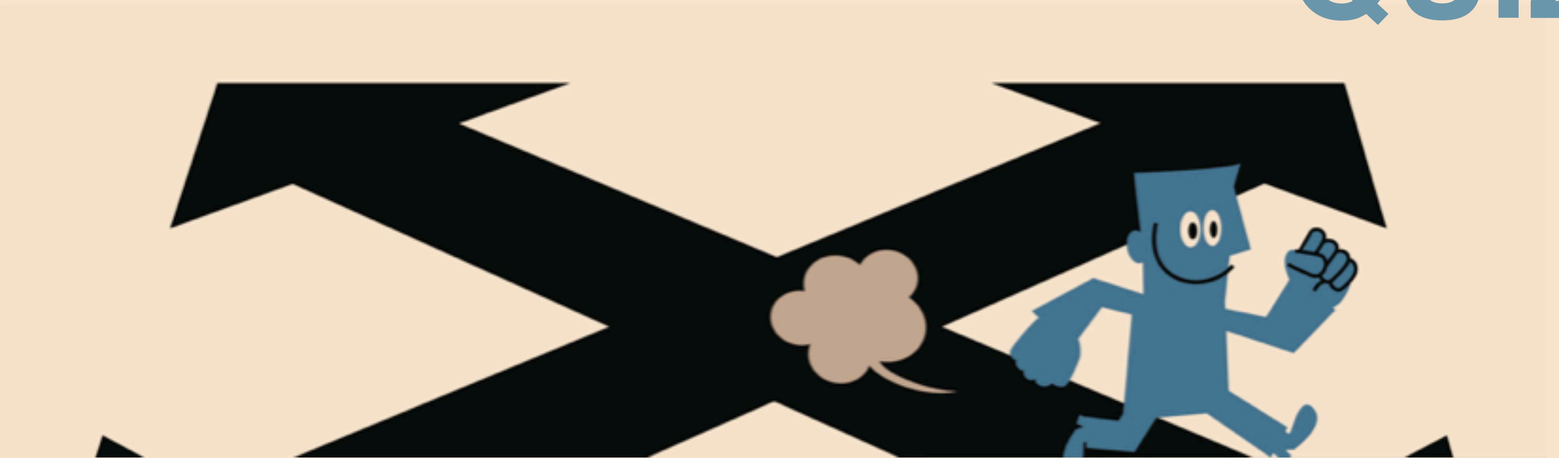
regardless of their **implementation technique**

Tail call optimization

Theorem

Recursive functions can become iterative processes (of constant stack size) if the **recursive call is the last call** in the procedure definition. That is there are **no unbound** references **left in the stack frame**

QUIZ 1





TAIL REC OR NO TAIL REC

Tailrec or no tailrec

```
fun {Sqrt Guess X}
   if {GoodEnough Guess X} then Guess
   else
   Guess1 = {Improve Guess X} in
      {Sqrt Guess X}
   end
end
```

Tailrec or no tailrec

```
fun {Sqrt Guess X}
   if {GoodEnough Guess X} then Guess
   else
   Guess1 = {Improve Guess X} in
      {Sqrt Guess X}
   end
end
```

Tailrec or no tailrec

```
fun {Sqrt Guess X}
   if {GoodEnough Guess X} then Guess
   else
   Guess1 = {Improve Guess X} in
      {Sqrt Guess X}
   end
end
```



Tailrec or no tailrec

```
fun {SumSqr Lst}
   local F in
      F = fun {$ Lst Accum}
         case Lst of nil then Accum
         [] X|Xs then {F Xs Accum+X*X}
      end
   {F Lst 0}
   end
end
```

Tailrec or no tailrec

```
fun {SumSqr Lst}
   local F in
      F = fun {$ Lst Accum}
         case Lst of nil then Accum
         [] X|Xs then {F Xs Accum+X*X}
      end
   end
   {F Lst 0}
   end
end
```

Tailrec or no tailrec

```
fun {SumSqr Lst}
   local F in
      F = fun {$ Lst Accum}
         case Lst of nil then Accum
         [] X|Xs then {F Xs Accum+X*X}
      end
   end
   {F Lst 0}
   end
end
```



Tailrec or no tailrec

```
fun {Exp X N}
   if N = 0 then 1
   else X * {Exp X N-1}
   end
end
```

Tailrec or no tailrec

```
fun {Exp X N}
   if N = 0 then 1
   else X * {Exp X N-1}
   end
end
```

Tailrec or no tailrec

```
fun {Exp X N}
   if N = 0 then 1
   else X * {Exp X N-1}
   end
end
```



Tailrec or no tailrec

```
fun {Real N Num}
   if N = 0 then Num
   else {Real N-1 Num+1.0}
   end
end
```

Tailrec or no tailrec

```
fun {Real N Num}
   if N = 0 then Num
   else {Real N-1 Num+1.0}
   end
end
```

Tailrec or no tailrec

```
fun {Real N Num}
   if N = 0 then Num
   else {Real N-1 Num+1.0}
   end
end
```



Tailrec or no tailrec

```
fun {Sorted L}
   case L of nil then true
   [] X|nil then true
   [] X|Y|Xs then {And X<Y {Sorted Y|Xs}}
   end
end
```

Tailrec or no tailrec

```
fun {Sorted L}
   case L of nil then true
   [] X|nil then true
   [] X|Y|Xs then {And X<Y {Sorted Y|Xs}}
   end
end
```

Tailrec or no tailrec

```
fun {Sorted L}
   case L of nil then true
   [] X|nil then true
   [] X|Y|Xs then {And X<Y {Sorted Y|Xs}}
   end
end
```



Tailrec or no tailrec

```
fun {Title S}
   case S of nil then ''''
   [] X|nil then ''''
   [] X|Y|Xs then
      if X = ' ' then
         {Append {Upper Y} {Title Xs}}
      else {Append {Upper X} Y|{Title Xs}}
      end
   end
end
```

Tailrec or no tailrec

```
fun {Title S}
   case S of nil then ''''
   [] X|nil then ''''
   [] X|Y|Xs then
      if X = ' ' then
         {Append {Upper Y} {Title Xs}}
      else {Append {Upper X} Y|{Title Xs}}
      end
   end
end
```

Tailrec or no tailrec

```
fun {Title S}
   case S of nil then ''''
   [] X|nil then ''''
   [] X|Y|Xs then
      if X = ' ' then
         {Append {Upper Y} {Title Xs}}
      else {Append {Upper X} Y|{Title Xs}}
      end
   end
end
```



From recursive to iterative processes

The **key** of iterative processes is that **recursive call must be the last call** in a procedure. This enables to **recover the procedures' environment**

One can transform a recursive process to an iterative one by means of (implicit) **state transformation**

From recursive to iterative processes

```
fun {Length Xs}
   case Xs of nil then 0
    [] Y|Ys then          1 + {Length Ys}
   end
end
```

From recursive to iterative processes

```
fun {Length Xs}
   case Xs of nil then 0
    [] Y|Ys then          1 + {Length Ys}
   end
end
```

process state

From recursive to iterative processes

```
fun {Length Xs}
   fun {Length Xs Accum}
      case Xs of nil then Accum
       [] Y|Ys then          {Length Ys 1+Accum}
      end
   end
   {Length Xs 0}
end
```

From recursive to iterative processes

```
fun {Length Xs}
   fun {Length Xs Accum}
      case Xs of nil then Accum
       [] Y|Ys then          {Length Ys 1+Accum}
      end
   end
   {Length Xs 0}
end
```

Implicit state

From recursive to iterative processes

```
fun {Length Xs}
   fun {Length Xs Accum}
      case Xs of nil then Accum
       [] Y|Ys then          {Length Ys 1+Accum}
      end
   end
   {Length Xs 0}
end
```

Implicit state

Nested function call

From recursive to iterative processes

```
fun {Length Xs}
   fun {Length Xs Accum}
      case Xs of nil then Accum
       [] Y|Ys then {Length Ys 1+Accum}
      end
   end
   {Length Xs 0}
end
```

Implicit state

Accumulative operation

Nested function call

2. Take the following process and write an equivalent iterative process

Do it yourself

```
fun {Exp X N}
   if N == 0 then 1
   else X * {Exp X N-1}
   end
end
```

2.

Do it yourself

```
fun {Exp X N}
   fun {Exp X N I}
      if N == 0 then I
      else {Exp X N-1 I*X}
      end
   end
   {Exp X N 1}
end
```

3. Take the following process and write an equivalent iterative process

Do it yourself

```
fun {Title S}
   case S of nil then ''''
   [] X|nil then ''''
   [] X|Y|Xs then
      if X == ' ' then
         {Append {Upper Y} {Title Xs}}
      else {Append {Upper X} Y|{Title Xs}}
      end
   end
end
```

3.

Do it yourself

```
fun {Title S}
  fun {Title S Up}
    case S of nil then {Reverse Up}
    [] X|nil then ''''
    [] X|Y|Xs then
     if X == ' ' then
      {Title Xs {Upper Y}|Up}
     else {Title Y|Xs {Upper X}|Up}
     end
    end
  end
  {Title S nil}
end
```

4. Take the following process and write an equivalent iterative process

Do it yourself

Factorial

4. Take the following process and write an equivalent iterative process

Do it yourself

Factorial

```
fun {Fact N}
      if N == 0 then 1
    else {Fact N-1} * N
    end
end
```

4.

Factorial

Do it yourself

```
local Fact
   fun {FactIter N I F}
      if I == N then F
      else {FactIter N I+1 F*(I+1)}
      end
   end
in
   fun {Fact N}
      {FactIter N 0 1}
   end
end
```

4.

Factorial

Do it yourself

```
fun {Fact N}
   fun {FactIter I F}
      if I == N then F
      else {FactIter I+1 F*(I+1)}
      end
   end
   {FactIter 0 1}
end
```

Factorial

Do it yourself

N is always in the scope of the inner function

```
fun {Fact N}
   fun {FactIter I F}
      if I == N then F
      else {FactIter I+1 F*(I+1)}
      end
   end
   {FactIter 0 1}
end
```

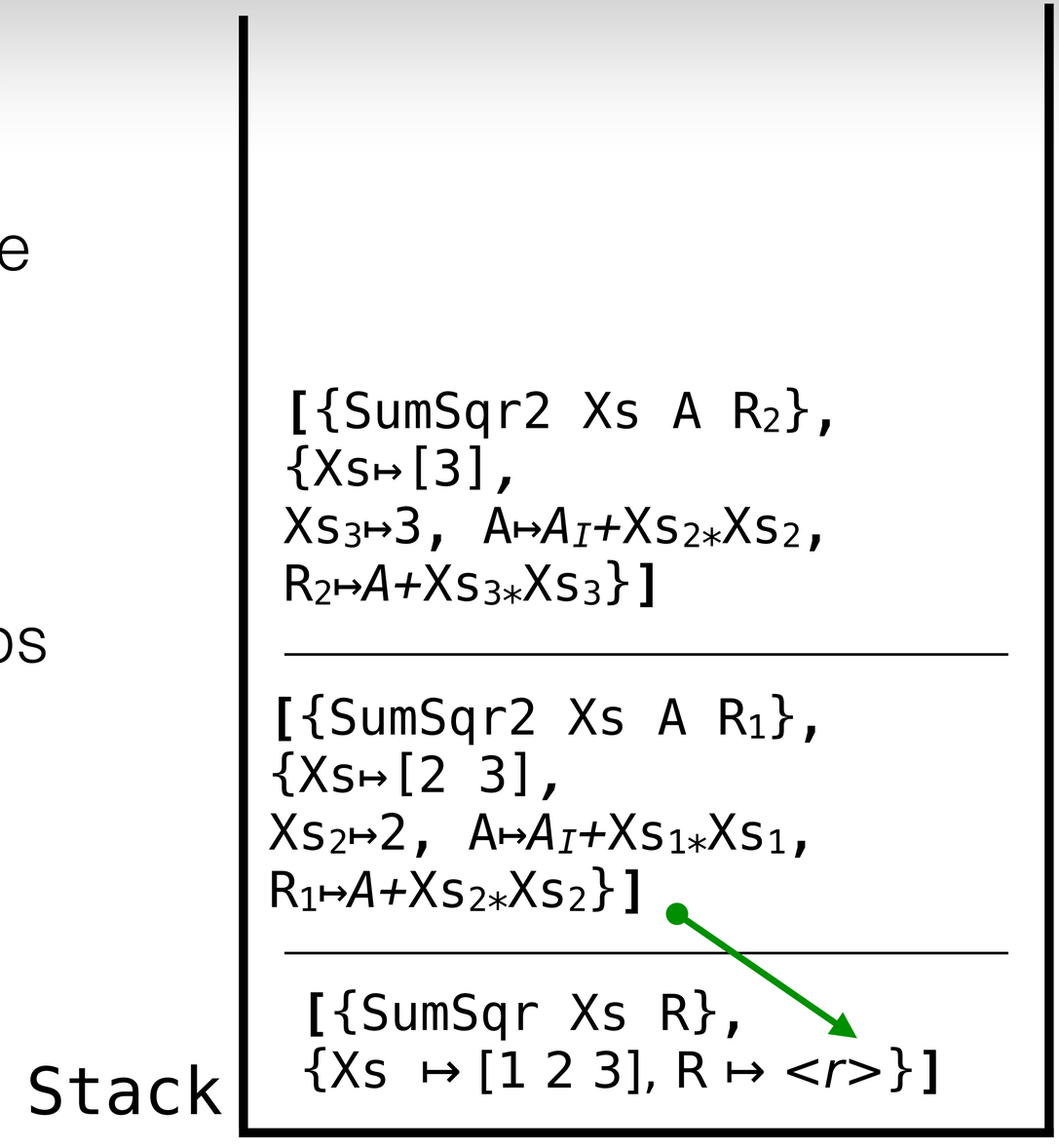
JAVA INTERLUDE



The Java deal

In Java several secure functions rely on stack counting. This means that optimizing for tail-calls is not possible (otherwise it breaks the secure functions)

Scala jumps through 10000000 hoops to get it done (@tailrec), but it is not possible to always optimize



TREES

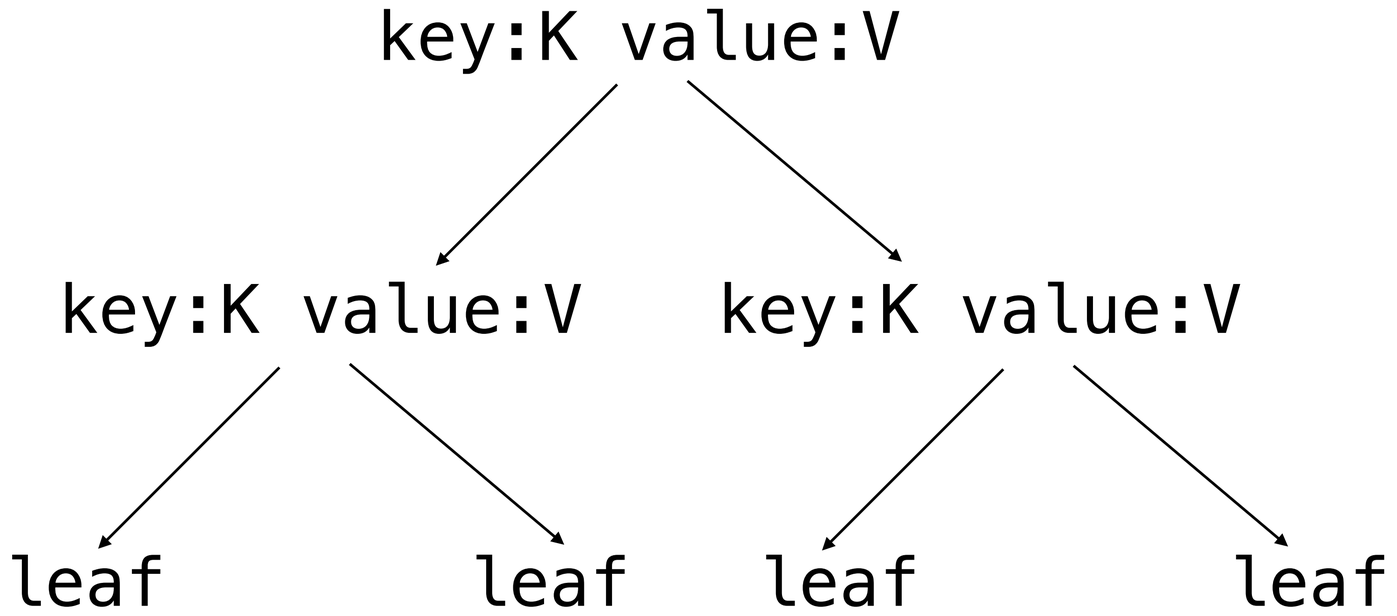


Trees

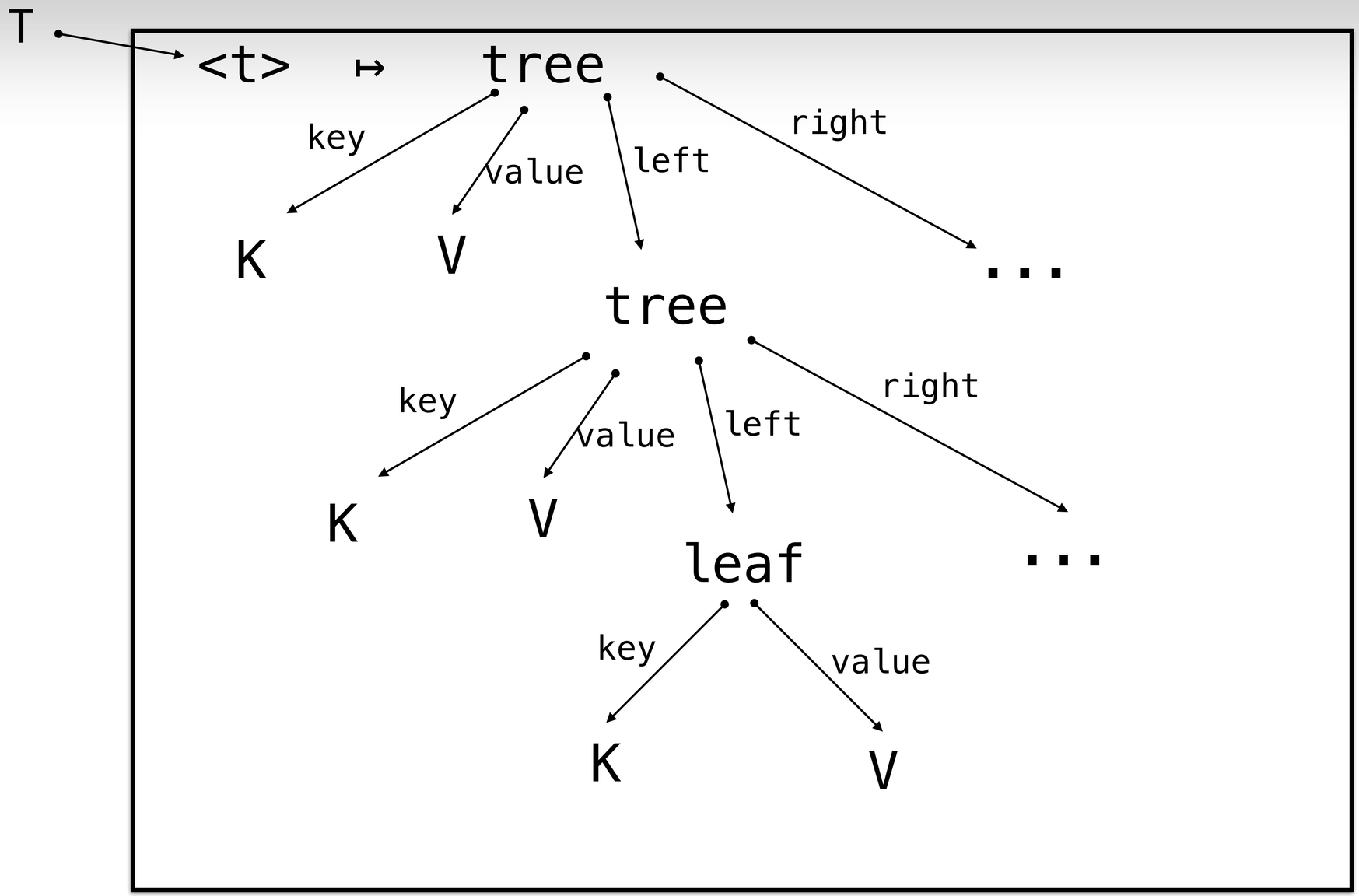
Trees are defined as a branching **recursive structure**

```
T ::= tree(key:K value:V left:Tl right:Tr)
    | leaf(key:K value:V)
    | nil
```

Binary tree, but extensible to trees of any arity

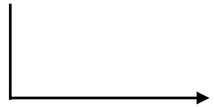


Trees


T
<t> ↦ tree
key
value
left
right
K
V
...
tree
key
value
left
right
K
V
leaf
...
key
value
K
V

Search trees

`{Lookup X T Dv ?V}`



Returns the value of the corresponding key

`{Insert X V T ?T₂}`



Now the tree contains the new (`X,V`) pair

`{Delete X T ?T₂}`



Now the tree doesn’t contain the pair (`X`, value)

Search trees

```
{Lookup X T Dv ?V}
```

Base case?
How to get it?

Search trees

```
{Lookup X T Dv ?V}
```

Base case?
How to get it?

```
fun {Lookup X T Dv}
   case T of leaf(key:Y value:V) then
      if X==Y then V else Dv end
   [] tree(key:Y value:V left:L right:R) andthen X==Y then
      V
   [] tree(key:Y value:V left:L right:R) then
      local Res Res2 in
         Res = {Lookup X L Dv}
         Res2 = {Lookup X R Dv}
         if Res\=Dv then Res elseif Res2\=Dv then Res2 else Dv end
      end
   end
end
```

Search trees

```
{Lookup X T Dv ?V}
```

Base case?
How to get it?

```
fun {Lookup X T Dv}
   case T of leaf(key:Y value:V) then
      if X==Y then V else Dv end
   [] tree(key:Y value:V left:L right:R) andthen X==Y then
      V
   [] tree(key:Y value:V left:L right:R) andthen X<Y then
      {Lookup X L Dv}
   [] tree(key:Y value:V left:L right:R) andthen X>Y then
      {Lookup X R Dv}
   end
end
```

Search trees

```
{Insert X V T ?T2}
```

Do it yourself

6.

Do it yourself

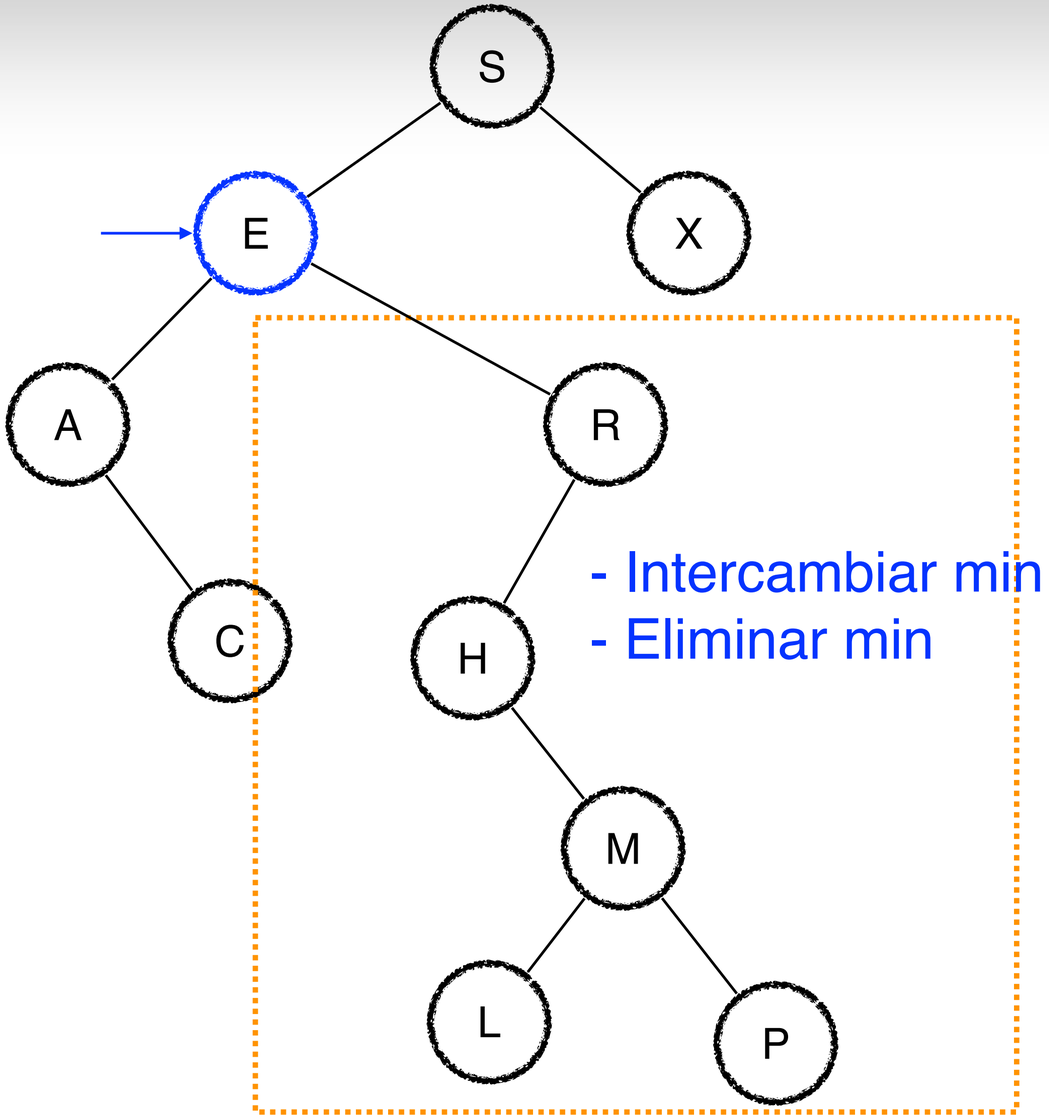
{Insert X V T ?T2}

```
fun {Insert X V T}
  case T of leaf(key:Y value:B) then
   if X==Y then leaf(key:Y value:V)
   elseif X<Y then tree(key:X value:V left:nil right:leaf(X V))
   else tree(key:Y value:V left:leaf(X V) right:nil)) end
  [] tree(key:Y value:B left:L right:R) andthen X==Y then
     tree(key:Y value:V left:L right:R)
  [] tree(key:Y value:B left:L right:R) andthen X<Y then
     tree(key:Y value:B left:{Insert X V L} right:R)
  [] tree(key:Y value:B left:L right:R) andthen X>Y then
     tree(key:Y value:B left:L right:{Insert X V R})
  end
end
```

Search trees

```
{Delete X T ?T2}
```

Eliminar **E**



Search trees

```
{Delete X T ?T2}
```

Do it yourself

7.

Do it yourself

{Delete X T ?T2}

```
fun {Delete X T}
  case T of leaf(key:Y value:B) then
   if X==Y then nil
   else leaf(key:Y value:B) end
  [] tree(key:Y value:B left:L right:R) andthen X==Y then
    case {RemoveSmallest R} of none then L
    [] Yp#Wp#Tp then tree(key:Yp value:Wp left:L right:Tp) end
  [] tree(key:Y value:V left:L right:R) andthen X<Y then
     tree(key:Y value:B left:{Delete X L} right:R)
  [] tree(key:Y value:V left:L right:R) andthen X>Y then
     tree(key:Y value:B left:L right:{Delete X R})
  end
end
```

7.

{Delete X T ?T2}

Do it yourself

```
fun {RemoveSmallest T}
   case T of leaf(key:K value:V) then none
   [] tree(key:X value:V T1 T2) then
      case {RemoveSmallest T2} of none then X#V#T2
      [] Xp#Vp#Tp then Xp#Vp#tree(key:X value:V T1 Tp)
      end
   end
end
```

DFS

Do it yourself

8.

{DFS T Xs}

Do it yourself

```
fun {DFS T Xs}
   case T of leaf(key:K value:V) then {Append Xs K#V}
   [] tree(key:X value:V left:L right:R) then
      {DFS L Xs}
      {DFS R {Append Xs X#V}}
   end
end
```

```
{BFS T Xs}
```

Do it yourself

9.

Do it yourself

```
{BFS T Xs}
```

```
fun {BFS T}
   fun {TreeInsert Q T}
      if T\=leaf then {Insert Q T} else Q end
   end
   proc {BFSQueue Q1}
      if {EmptyQueue Q1} then skip
      else X Q2={Delete Q1 X}
         X = tree(key:K value:V left:L right:R)
      in
         {Show K#V}
         {BFSQueue {TreeInsert {TreeInsert Q2 R} L}}
      end
   end
in
      {BFSQueue {TreeInsert {NewQueue} T}}
end
```

CAN ALL RECURSIVE CALLS BE TAIL RECURSIVE



NO

Recursive functions

Tak is a function famously defined by Ikuo Takeuchi to benchmark tail-recursion in programming languages

Tak **cannot** turn into a tail recursive call

Tak

$$\tau(x,y,z) = \begin{cases} \tau(\tau(x-1,y,z),\tau(y-1,z,x),\tau(z-1,x,t)) & \text{if } y < x \\ z & \text{otherwise} \end{cases}$$

Tak

$$\tau(x,y,z) = \begin{cases} \tau(\tau(x-1,y,z),\tau(y-1,z,x),\tau(z-1,x,t)) & \text{if } y < x \\ z & \text{otherwise} \end{cases}$$

```
fun {Tak X Y Z}
   if Y < X then
    {Tak {Tak X-1 Y Z} {Tak Y-1 Z X} {Tak Z-1 X Y}}
   else Z
   end
end
```

Tak

$$\tau(x,y,z) = \begin{cases} \tau(\tau(x-1,y,z),\tau(y-1,z,x),\tau(z-1,x,t)) & \text{if } y < x \\ z & \text{otherwise} \end{cases}$$

```
fun {Tak X Y Z}
   if Y < X then
    {Tak {Tak X-1 Y Z} {Tak Y-1 Z X} {Tak Z-1 X Y}}
   else Z
   end
end
```

The same trick we always use isn't obvious anymore

Quicksort

```
fun {Quick Xs}
  fun {Less Y Ys}
   if Y==nil orlese Ys==nil then nil
   elseif Y < {Nth Ys 1} then {Less Y {Drop Ys 1}}
   else {Nth Ys 1}|{Less Y {Drop Ys 1}} end
  end
  fun {Right Y Ys}
   if Y==nil orlese Ys==nil then nil
   elseif Y => {Nth Ys 1} then {Right Y {Drop Ys 1}}
   else {Nth Ys 1}{Right Y {Drop Ys 1}} end
  end
  case Xs of nil then nil
  [] X|Xr then
    {Append {Quick {Left X Xr}} X|nil {Quick {Right X Xr}}
  end
end
```

Quicksort

```
fun {Quick Xs}
  fun {Less Y Ys}
   if Y==nil orlese Ys==nil then nil
   elseif Y < {Nth Ys 1} then {Less Y {Drop Ys 1}}
   else {Nth Ys 1}|{Less Y {Drop Ys 1}} end
  end
  fun {Right Y Ys}
   if Y==nil orlese Ys==nil then nil
   elseif Y => {Nth Ys 1} then {Right Y {Drop Ys 1}}
   else {Nth Ys 1}{Right Y {Drop Ys 1}} end
  end
  case Xs of nil then nil
  [] X|Xr then
    {Append {Quick {Left X Xr}} X|nil {Quick {Right X Xr}}
  end
end
```

One of the two {Quick …} calls can be tailrec, not both

Summary

- Recursion has to do with looping more than anything else i.e., make a calculation by the repetitive application of a function.
- There are two flavors to **recursion**
  - If the stack trace of the process **grows** unmanaged, we say the **process is recursive**
  - If the stack trace of the process is **constant**, we say the **process is iterative**
- Recursion is ideal for recursive structures